



MACHINE LEARNING IN BIOINFORMATICS

Part 7: Hidden Markov Models

(Adapted slides from

http://bix.ucsd.edu/bioalgorithms/presentations/Ch11_HMM.ppt

and book R. Durbin, S. R. Eddy, A. Krogh, G. Mitchison:
Biological sequence analysis. Cambridge University Press,
1998)

František Mráz

KSVI MFF UK

Outline



1. **CG-islands**
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm – the most probable path
5. Outcome likelihood
6. Forward-Backward Algorithm
7. HMM Parameter Estimation
8. Viterbi training
9. Baum-Welch algorithm
10. Applications of HMM in Biology

Dinucleotide frequency – CG-Islands



- Consider all 2-mers in a sequence
 $\{AA, AC, AG, AT, CA, CC, CG, CT, GA, GC, GG, GT, TA, TC, TG, TT\}$
- Given 4 nucleotides: each with probability of occurrence $\sim \frac{1}{4}$.
Thus, expected probability of occurrence of a dinucleotide is $\sim \frac{1}{16}$.
- However, the frequencies of dinucleotides in DNA sequences vary widely.
- In particular, frequency of **CG** is typically $< \frac{1}{16}$

Note: In literature, CG-islands are often called CpG-islands as to distinguish it from C and G rich segments.

Example



- From a 291829-base sequence

frequency		frequency	
AA	0.120214646984	GA	0.056108392614
AC	0.055409350713	GC	0.037792809463
AG	0.068848773935	GG	0.043357731266
AT	0.083425853585	GT	0.046828954041
CA	0.074369148950	TA	0.077206436668
CC	0.044927148868	TC	0.056207766218
CG	0.008179475581	TG	0.063698479926
CT	0.066857875186	TT	0.096567155996

- Expected value 0.0625
- The frequency of CG is 7 times smaller than expected!

Why CG-Islands?



- **CG** is the least frequent dinucleotide because **C** in CG is easily *methyalted* (that is, an H-atom is replaced by a **CH₃**-group) and the methyl-**C** has the tendency to mutate into **T** afterwards
- However, the methylation is suppressed around genes and transcription factor regions in a genome. So, **CG** appears at *relatively* higher frequency within these important areas called **CG-islands**
- Finding the **CG** islands within a genome is among the most reliable gene finding approaches
- **Classical definition:** A **CG** island is DNA sequence of length about 200bp with a **C+G** content of 50% and a ratio of observed-to-expected number of **CG**'s that is above 0.6. (Gardiner-Garden & Frommer, 1987)

Problems



1. **Discrimination problem:** Given a short segment of genomic sequence. How can we decide whether this segment comes from a CG-island or not?

→ Markov Model

2. **Localization problem:** Given a long segment of genomic sequence. How can we find all contained CG-islands?

→ Hidden Markov Model

Markov Model



Definition: A (time-homogeneous) **Markov model** (of order 1) is a system $M = (Q, A)$ consisting of

$Q = \{s_1, \dots, s_k\}$: a finite set of states and

$A = (a_{kl})$: a $|Q| \times |Q|$ matrix of probability of changing from state s_k to state s_l . $P(x_{i+1} = s_l, x_i = s_k) = a_{kl}$ with $\sum_{l \in S} a_{kl} = 1$ for all $k \in S$.

Definition: A **Markov chain** is a chain $x_0, x_1, \dots, x_n, \dots$ of random variables, which take states in the state set Q such that

$P(x_n = s \mid \bigcap_{j < n} x_j) = P(x_n = s \mid x_{n-1})$ is *true* for all $n > 0$ and $s \in S$.

Definition: A Markov chain is called **homogeneous**, if the probabilities are not dependent on n . (At any time i the chain is in a specific state x_i and at the tick of a clock the chain changes to state x_j according to the given transition probabilities.)

Example



- Weather in Prague, daily at midday:
 - Possible states are **rain**, **sun** or **clouds**.
 - Transition probabilities:

	r	s	c
r	0.2	0.3	0.5
s	0.2	0.6	0.2
c	0.3	0.3	0.4

- A Markov chain would be the observation of the weather:
...rrrrrrccsssssscscscrrcrssss...
- Types of questions that the model can answer:
 1. If it is sunny today, what is the probability that the sun will shine for the next seven days?
 2. How large is the probability, that it will rain for a month?

Modeling the Begin and End states



- We must specify the initialization of the chain – an initial probability $P(x_1)$ of starting in a particular state. We can add a begin state to the model that is labeled '*Begin*' and add this to the states set. We will always assume that $x_0 = \textit{Begin}$ holds. Then the probability of the first state in the Markov chain is

$$P(x_1 = s) = a_{\textit{Begin},s} = P(s),$$

where $P(s)$ denotes the background probability of state s .

- Similarly, we explicitly model the end of the sequence using an end state '*End*'. Thus, the probability that we end in state t is

$$P(\textit{End} | x_n = t) = a_{t,\textit{End}}.$$

Probability of Markov chains



- Given a sequence of states $\mathbf{x} = x_1, x_2, x_3, \dots, x_L$. What is the probability that a Markov chain will step through precisely this sequence of states?

$$\begin{aligned} P(\mathbf{x}) &= P(x_1, x_2, \dots, x_L) \\ &= P(x_1, \dots, x_{L-1})P(x_L | x_1, \dots, x_{L-1}) \\ &= P(x_1)P(x_2 | x_1) \cdots P(x_{L-1} | x_{L-2}, \dots, x_1)P(x_L | x_{L-1}, \dots, x_1) \\ &\quad \text{[by repeated application of } P(X, Y) = P(X|Y)P(Y)\text{]} \\ &= P(x_1)P(x_2 | x_1) \cdots P(x_{L-1} | x_{L-2})P(x_L | x_{L-1}) \\ &= P(x_1) \prod_{i=2}^L a_{x_{i-1}, x_i} = \prod_{i=1}^L a_{x_{i-1}, x_i} \end{aligned}$$

↑
If $x_0 = \textit{Begin}$

Example



- # Markov chain that generates CpG islands
- # (Source: DEKM98, p 50)
- # Number of states:
- 6
- # State labels (*=Begin, +=End):
- A C G T * +
- # Transition matrix:
- 0.1795 0.2735 0.4255 0.1195 0 0.002
- 0.1705 0.3665 0.2735 0.1875 0 0.002
- 0.1605 0.3385 0.3745 0.1245 0 0.002
- 0.0785 0.3545 0.3835 0.1815 0 0.002
- 0.2495 0.2495 0.2495 0.2495 0 0.002
- 0.0000 0.0000 0.0000 0.0000 0 1.000

Transition matrices are generally calculated from training sets.

- In our case the transition matrix $\mathbf{P}^+$ for a DNA sequence that comes from a CG-island, is determined as follows:

$$p_{st}^+ = \frac{c_{st}^+}{\sum_{t'} c_{st'}^+}$$

- where c_{st} is the number of positions in a training set of CG-islands at which the state s is followed by the state t .

Markov chains for CG-islands and non-CG-islands



```
# Markov chain for CpG islands
# Number of states:
4
# State labels:
A C G T
# Transition matrix P+:
.1795 .2735 .4255 .1195
.1705 .3665 .2735 .1875
.1605 .3385 .3745 .1245
.0785 .3545 .3835 .1815
```

model⁺

```
# Markov chain for non-CpG islands
# Number of states:
4
# State labels:
A C G T
# Transition matrix P-:
.2995 .2045 .2845 .2095
.3215 .2975 .0775 .0775
.2475 .2455 .2975 .2075
.1765 .2385 .2915 .2915
```

model⁻

Begin and End
states are
missing?!

Solving Problem 1 – discrimination



- Given a short sequence $\mathbf{x} = (x_1, x_2, \dots, x_L)$. Does it come from a CG-island ($model^+$)?

$$P(\mathbf{x} | model^+) = \prod_{i=1}^L a_{x_{i-1}, x_i}^+$$

- Or does it not come from a non-CG-island ($model^-$)?

$$P(\mathbf{x} | model^-) = \prod_{i=1}^L a_{x_{i-1}, x_i}^-$$

- We calculate the log-odds ratio

$$S(\mathbf{x}) = \log \frac{P(\mathbf{x} | model^+)}{P(\mathbf{x} | model^-)} = \sum_{i=1}^L \log \left(\frac{a_{x_{i-1}, x_i}^+}{a_{x_{i-1}, x_i}^-} \right) = \sum_{i=1}^L \beta_{x_{i-1}, x_i}$$

with β_{XY} being the log-likelihood ratios of corresponding transition probabilities. For the transition matrices above we calculate for example $\beta_{AA} = \log \left(\frac{0.1795}{0.2995} \right)$. Often the base 2 logarithm is used, in which case the unit is in bits.

Solving Problem 1 – discrimination cont



- If $model^+$ and $model^-$ differ substantially then a typical CG-island should have a higher probability within the $model^+$ than in the $model^-$. The log-odds ratio should become positive.
- Generally, we could use a threshold value c^* and a test function to determine whether a sequence x comes from a CG-island:

$$\phi^*(x) := \begin{cases} 1 & \text{if } S(x) > c^* \\ 0 & \text{if } S(x) \leq c^* \end{cases}$$

Will c^* be positive or negative?

where $\phi^*(x) = 1$ indicates that x comes from a CG-island.

- Such a test is called Neyman-Pearson-Test.

Outline



1. CG-islands
2. **The “Fair Bet Casino”**
3. Hidden Markov Model
4. Decoding Algorithm – the most probable path
5. Outcome likelihood
6. Forward-Backward Algorithm
7. HMM Parameter Estimation
8. Viterbi training
9. Baum-Welch algorithm
10. Applications of HMM in Biology

CG Islands and the “Fair Bet Casino”



- The problem of localizations of CG-islands can be modeled after a problem named ***“The Fair Bet Casino”***
- The game is to flip coins, which results in only two possible outcomes: **H**ead or **T**ail.
- The **F**air coin will give **H**eads and **T**ails with same probability $\frac{1}{2}$.
- The **B**iased coin will give **H**eads with prob. $\frac{3}{4}$.
- Thus, we define the probabilities:
 - $P(H|F) = P(T|F) = \frac{1}{2}$
 - $P(H|B) = \frac{3}{4}, \quad P(T|B) = \frac{1}{4}$
 - The crooked dealer changes between Fair and Biased coins with probability 10%

The Fair Bet Casino Problem



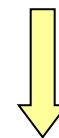
- **Input:** A sequence $\mathbf{x} = x_1 x_2 x_3 \cdots x_n$ of coin tosses made by two possible coins (**F** or **B**).
- **Output:** A sequence $\boldsymbol{\pi} = \pi_1 \pi_2 \pi_3 \cdots \pi_n$, with each π_i being either **F** or **B** indicating that x_i is the result of tossing the **F**air or **B**iased coin respectively.

Fair Bet Casino Problem

Any observed outcome of coin tosses could have been generated by any sequence of states!

⇒ ***Ill formulated problem!***

Need to incorporate a way to grade different sequences differently.



Decoding Problem

$P(x \mid \text{fair coin})$ vs. $P(x \mid \text{biased coin})$



- Suppose first that dealer never changes coins. Some definitions:
 - $P(x \mid \text{fair coin})$: probability of the dealer using the **F** coin and generating the outcome **x**.
 - $P(x \mid \text{biased coin})$: prob. of the dealer using the **B** coin and generating outcome **x**.

$P(\mathbf{x} \mid \text{fair coin})$ VS. $P(\mathbf{x} \mid \text{biased coin})$



$$\begin{aligned} P(\mathbf{x} \mid \text{fair coin}) &= P(x_1 \cdots x_n \mid \text{fair coin}) \\ &= \prod_{i=1}^n p(x_i \mid \text{fair coin}) = \left(\frac{1}{2}\right)^n \end{aligned}$$

$$\begin{aligned} P(\mathbf{x} \mid \text{biased coin}) &= P(x_1 \cdots x_n \mid \text{biased coin}) \\ &= \prod_{i=1}^n p(x_i \mid \text{biased coin}) = \left(\frac{3}{4}\right)^k \left(\frac{1}{4}\right)^{n-k} \end{aligned}$$

k – the number of Heads in $\mathbf{x}$.

$P(x \mid \text{fair coin})$ vs. $P(x \mid \text{biased coin})$



$$P(x \mid \text{fair coin}) = P(x \mid \text{biased coin})$$

$$\left(\frac{1}{2}\right)^n = \frac{3^k}{4^n}$$

$$2^n = 3^k$$

$$n = k \log_2 3$$

- when $k < n / \log_2 3$ ($k \sim 0.63n$), the dealer most likely used the fair coin
- when $k > n / \log_2 3$, he most likely used the biased coin

Computing Log-odds Ratio in Sliding Windows



$$x_1 x_2 \boxed{x_3 x_4 x_5 x_6 x_7} x_8 \dots x_n$$

Consider a *sliding window* of the outcome sequence. Find the log-odds for this short window.

$$\log_2 \frac{P(\text{window} \mid \text{fair coin})}{P(\text{window} \mid \text{biased coin})}$$



Disadvantages:

- the length of CG-island is not known in advance
- different lengths windows may classify the same position differently

Outline



1. CG-islands
2. The “Fair Bet Casino”
- 3. Hidden Markov Model**
4. Decoding Algorithm – the most probable path
5. Outcome likelihood
6. Forward-Backward Algorithm
7. HMM Parameter Estimation
8. Viterbi training
9. Baum-Welch algorithm
10. Applications of HMM in Biology

Hidden Markov Model (HMM)



- Can be viewed as an abstract machine with k **hidden** states that emits symbols from an alphabet Σ .
- Each state has its own probability distribution, and the machine switches between states according to this probability distribution.
- While in a certain state, the machine makes 2 decisions:
 - What symbol – from the alphabet Σ – should I emit?
 - What state should I move to next?
- Observer can see the emitted symbols of an HMM but *have no ability to know which state the HMM is currently in*
- Thus, the goal is to *infer the most likely hidden states of an HMM* based on the given sequence of emitted symbols

HHHTHTHHTTTTHTHTHTHHHTHTHTHT

BBBFFFFFFFFFFFFFFFFFFFFBBBFFFFFF?

HMM Parameters

$M(\Sigma, Q, A, E)$



Σ : a set of emission characters.

Ex.: $\Sigma = \{H, T\}$ for coin tossing

$\Sigma = \{1, 2, 3, 4, 5, 6\}$ for dice tossing

Q : a set of hidden states, each emitting symbols from Σ .

$Q = \{F, B\}$ for coin tossing

$A = (a_{kl})$: a $|Q| \times |Q|$ matrix of probability of changing from state k to state l .

$$a_{FF} = 0.9 \quad a_{FB} = 0.1$$

$$a_{BF} = 0.1 \quad a_{BB} = 0.9$$

$E = (e_k(b))$: a $|Q| \times |\Sigma|$ matrix of probability of emitting symbol b while being in state k .

$$e_F(0) = \frac{1}{2} \quad e_F(1) = \frac{1}{2} \quad 0 = Tail$$

$$e_B(0) = \frac{1}{4} \quad e_B(1) = \frac{3}{4} \quad 1 = Head$$

HMM for Fair Bet Casino



- The *Fair Bet Casino* in *HMM* terms:
 $\Sigma = \{0, 1\}$ (0 for **T**ails and 1 **H**eads)
 $Q = \{F, B\}$ – *F* for Fair & *B* for Biased coin.

Transition Probabilities *A*

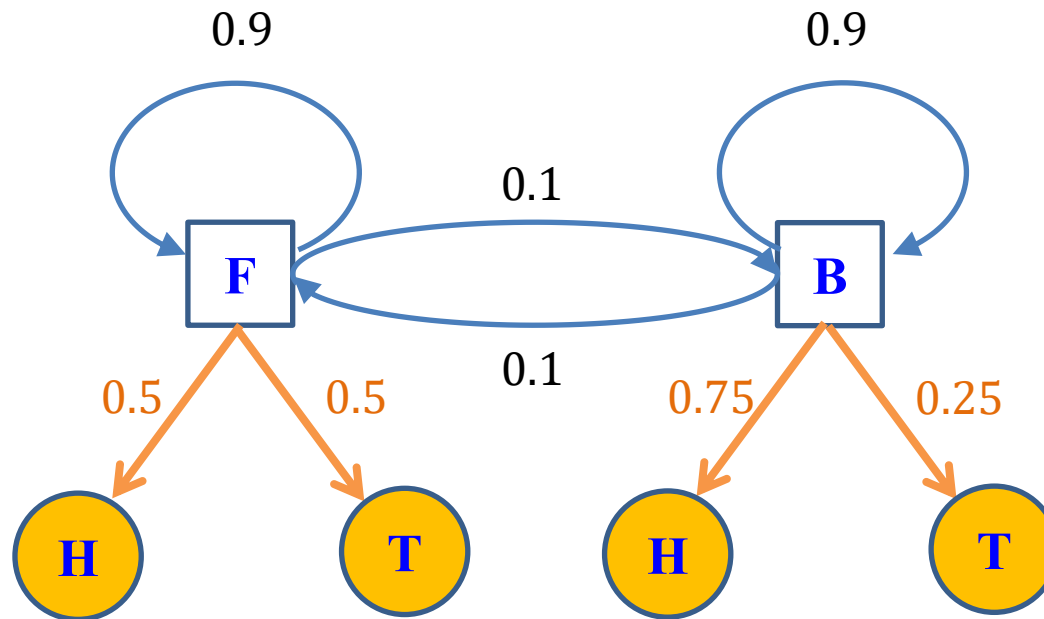
	Fair	Biased
Fair	$a_{FF} = 0.9$	$a_{FB} = 0.1$
Biased	$a_{BF} = 0.1$	$a_{BB} = 0.9$

Emission Probabilities *E*

	Tails(0)	Heads(1)
Fair	$e_F(0) = \frac{1}{2}$	$e_F(1) = \frac{1}{2}$
Biased	$e_B(0) = \frac{1}{4}$	$e_B(1) = \frac{3}{4}$

HMM for Fair Bet Casino

(cont'd)



HMM model for the *Fair Bet Casino* Problem

Hidden Paths



- A *path* $\pi = \pi_1 \dots \pi_n$ in the HMM is defined as a sequence of states.
- Consider path $\pi = \text{FFFB BBBB FFF}$ and sequence $x = 01011101001$
- $P(x, \pi)$ is the probability that HMM follows the hidden path π and emits the sequence x

$$\sum_{\text{all possible } x} \sum_{\text{all possible } \pi} P(x, \pi) = 1$$

- $P(x | \pi)$ is the conditional probability that HMM emits the sequence x when following the hidden path π

$$\sum_{\text{all possible } x} P(x | \pi) = 1$$

Computing $P(x, \pi)$



- A *path* $\pi = \pi_1 \dots \pi_n$ in the HMM is defined as a sequence of states.

$$P(x, \pi) = P(x | \pi) \cdot P(\pi)$$

Probability that x_i was emitted being in state π_i

x	0	1	0	1	1	1	0	1	0	0	1
π	F	F	F	B	B	B	B	B	F	F	F
$P(x_i \pi_i)$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{3}{4}$	$\frac{1}{4}$	$\frac{3}{4}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$
$P(\pi_{i-1} \rightarrow \pi_i)$	$\frac{1}{2}$	$\frac{9}{10}$	$\frac{9}{10}$	$\frac{1}{10}$	$\frac{9}{10}$	$\frac{9}{10}$	$\frac{9}{10}$	$\frac{9}{10}$	$\frac{1}{10}$	$\frac{9}{10}$	$\frac{9}{10}$

Transition from the state *begin*

Transition probability from state π_{i-1} to state π_i

$P(\mathbf{x}, \boldsymbol{\pi})$ Calculation



- $P(\mathbf{x}, \boldsymbol{\pi})$: Probability that the sequence $\mathbf{x} = x_1 x_2 \dots x_n$ was generated by the path $\boldsymbol{\pi} = \pi_1 \pi_2 \dots \pi_n$:

$$P(\mathbf{x}, \boldsymbol{\pi}) = P(\boldsymbol{\pi}) \cdot P(\mathbf{x} \mid \boldsymbol{\pi})$$

$$= P(\pi_1) \prod_{i=2}^n P(\pi_{i-1} \rightarrow \pi_i) \cdot \prod_{i=1}^n P(x_i \mid \pi_i)$$

$$= \prod_{i=1}^n P(\pi_{i-1} \rightarrow \pi_i) \cdot \prod_{i=1}^n P(x_i \mid \pi_i)$$

$\pi_0 = \text{Begin}$

$$= \prod_{i=1}^n P(\pi_{i-1} \rightarrow \pi_i) \cdot P(x_i \mid \pi_i)$$

$$= \prod_{i=1}^n a_{\pi_{i-1}, \pi_i} \cdot e_{\pi_i}(x_i)$$

Outline



1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. **Decoding Algorithm – the most probable path**
5. Outcome likelihood
6. Forward-Backward Algorithm
7. HMM Parameter Estimation
8. Viterbi training
9. Baum-Welch algorithm
10. Applications of HMM in Biology

Decoding Problem

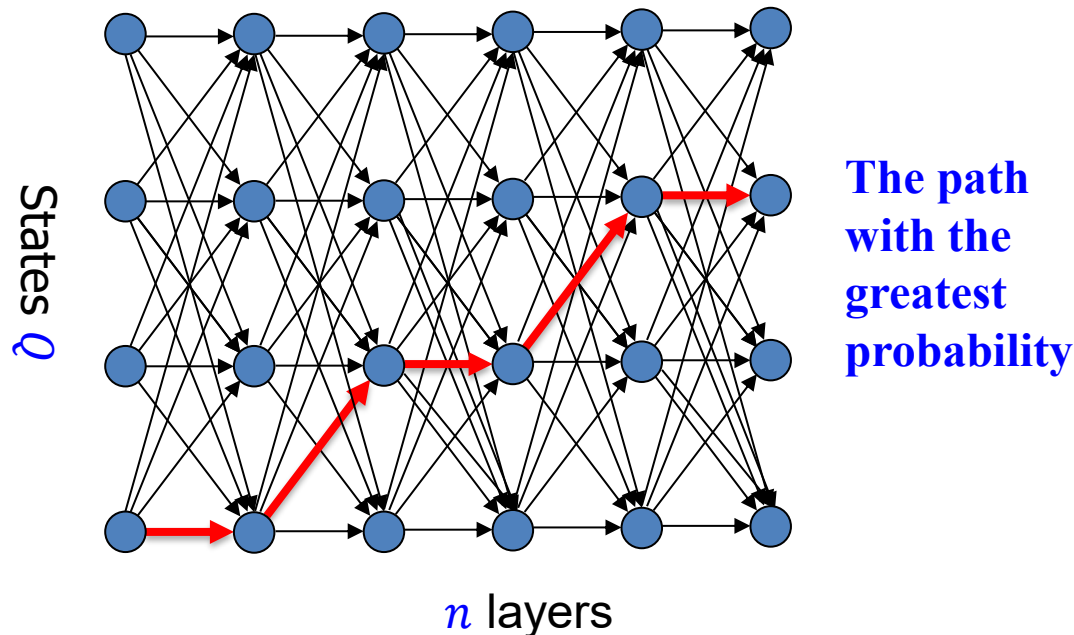


- **Goal:** Find an optimal hidden path of states given observations.
 - **Input:** Sequence of observations $\mathbf{x} = x_1 \dots x_n$ generated by an HMM $M(\Sigma, Q, A, E)$
 - **Output:** A path π that maximizes $P(\mathbf{x}, \pi)$ over all possible paths.
- ➡ *Solves Problem 2 – localization*

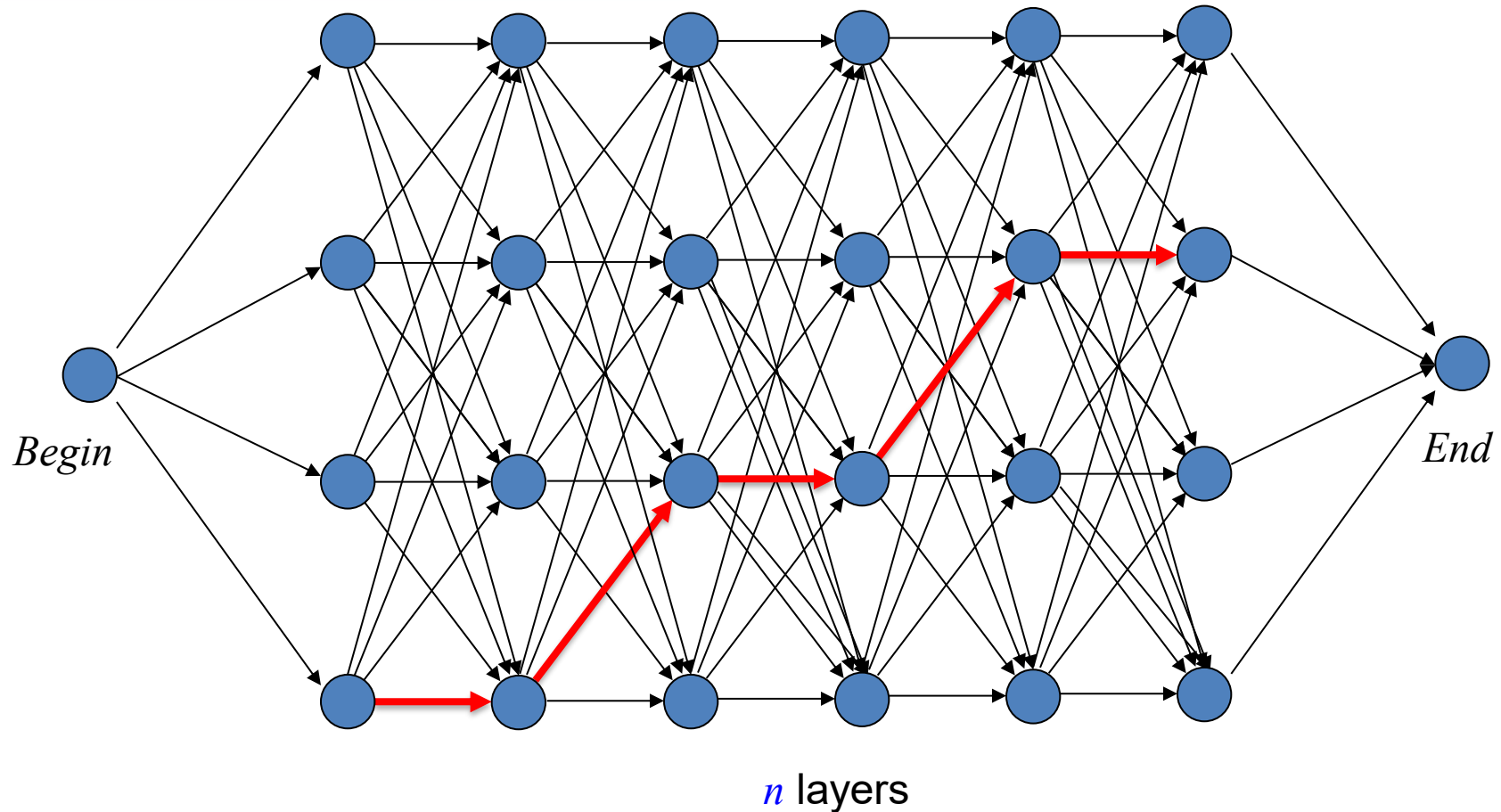
Edit Graph for Decoding Problem



- Andrew Viterbi used a grid model to solve the *Decoding Problem*.
- Every choice of $\pi = \pi_1 \cdots \pi_n$ corresponds to a path in a graph.
- The only valid direction in the graph is *eastward*.
- This graph has $|Q|^2 (n - 1)$ edges.



Edit Graph for Decoding Problem

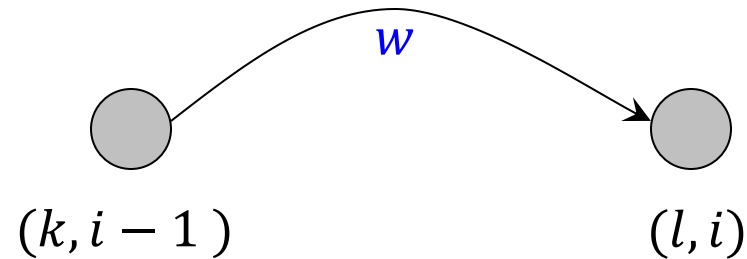


Decoding Problem as Finding a Longest Path in a DAG



- The *Decoding Problem* is reduced to finding a longest path in the *directed acyclic graph (DAG)* above.
- Notes: the length of the path is defined as the **product** of its edges' weights, not the **sum**.
- Every path in the graph has the probability $P(x, \pi)$.
- The Viterbi algorithm finds the path that maximizes $P(x, \pi)$ among all possible paths.
- The Viterbi algorithm runs in $O(n|Q|^2)$ time.

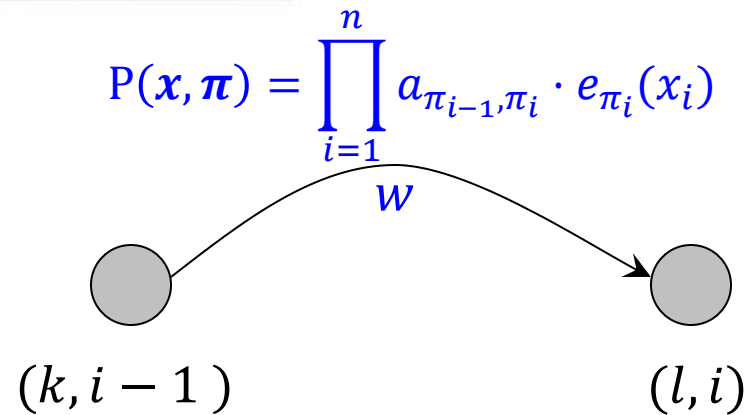
Decoding Problem: weights of edges



The weight w is given by:

???

Decoding Problem: weights of edges



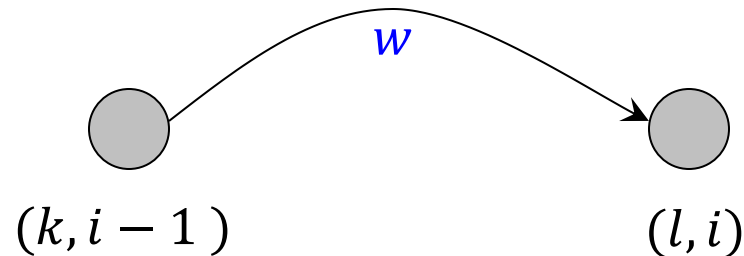
The weight w is given by:

??

Decoding Problem: weights of edges

$$P(\mathbf{x}, \boldsymbol{\pi}) = \prod_{i=1}^n a_{\pi_{i-1}, \pi_i} \cdot e_{\pi_i}(x_i)$$

i -th term = $a_{\pi_{i-1}, \pi_i} \cdot e_{\pi_i}(x_i)$



The weight w is given by:

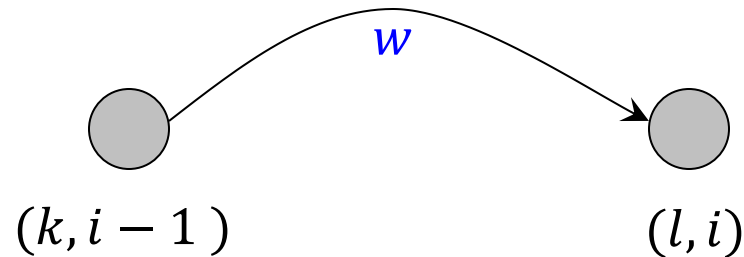
?

Each weight is a factor in the product

Decoding Problem: weights of edges

$$P(\mathbf{x}, \boldsymbol{\pi}) = \prod_{i=1}^n a_{\pi_{i-1}, \pi_i} \cdot e_{\pi_i}(x_i)$$

i -th term $= a_{\pi_{i-1}, \pi_i} \cdot e_{\pi_i}(x_i) = a_{k,l} \cdot e_l(x_i)$ for $\pi_{i-1} = k, \pi_i = l$



The weight $w = a_{kl} \cdot e_l(x_i)$

Decoding Problem and Dynamic Programming



Let s_{li} denote the probability of the most likely path generating the prefix $x_1, \dots, x_i$ and ending in state l

$$\begin{aligned} s_{li} &= \max_{k \in Q} \{s_{k,i-1} \cdot \text{weight of the edge between } (k, i-1) \text{ and } (l, i)\} = \\ &= \max_{k \in Q} \{s_{k,i-1} \cdot a_{kl} \cdot e_l(x_i)\} \\ &= e_l(x_i) \cdot \max_{k \in Q} \{s_{k,i-1} \cdot a_{kl}\} \end{aligned}$$

Decoding Problem (cont'd)



- **Initialization:**
 - $s_{Begin,0} = 1$
 - $s_{k,0} = 0$ for $k \neq Begin$.
- Let π^* be the optimal path. Then,

$$P(x, \pi^*) = \max_{k \in Q} \{s_{k,n}\}$$

Viterbi Algorithm



- The value of the product can become extremely small, which leads to underflowing → use $\log$ value instead.
- **Goal:** Find an optimal hidden path of states given observations.
- **Input:** Sequence of observations $\mathbf{x} = x_1, \dots, x_n$ generated by an HMM $M(\Sigma, Q, A, E)$
- **Output:** A path that maximizes $P(\mathbf{x}, \boldsymbol{\pi})$ over all possible paths $\boldsymbol{\pi}$.
- **Initialization:**
 $s_{Begin,0} = \log 1 = 0$
 $s_{k,0} = \log 0 = -\infty$ for $k \neq Begin$.
- **Iterate:**
For $i = 1$ to n
For $l = 1$ to $|Q|$
 $s_{l,i} = \log e_l(x_i) + \max_{k \in Q} \{s_{k,i-1} + \log a_{kl}\}$ // note where the maximum was achieved
- **Output:** the sequence $\pi_1, \dots, \pi_n$ such that
$$s_{\pi_n^*,n} = \max_{k \in Q} s_{k,n} = \max_{\boldsymbol{\pi}} \sum_{i=1,\dots,n} (\log e_{\pi_i} + \log a_{\pi_{i-1},\pi_i})$$

Outline



1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm – the most probable path
5. **Outcome likelihood**
6. Forward-Backward Algorithm
7. HMM Parameter Estimation
8. Viterbi training
9. Baum-Welch algorithm
10. Applications of HMM in Biology

Outcome Likelihood Problem



- **Input:** A string $x = x_1x_2 \cdots x_n$ emitted by an HMM $M(\Sigma, Q, A, E)$
- **Output:** The probability $P(x)$ that the HMM emits x
- Can we compute $P(x)$ by making a single change in the Viterbi algorithm?

$$P(\pi) = \sum_{\text{all possible } x} P(x, \pi)$$

$$P(x) = \sum_{\text{all possible } \pi} P(x, \pi)$$

$$P(x) = \sum_{\text{all possible } \pi} \text{product weight of } \pi$$

$$s_{End,i} = \max_{\text{all possible } \pi} \text{product weight of } \pi$$

Outcome Likelihood Problem



- **Viterbi algorithm:** s_{li} denote the logarithm of the probability of the most likely path generating the prefix $x_1, \dots, x_i$ and ending in state l
- **Outcome likelihood problem:** o_{li} denote the probability that the prefix $x_1, \dots, x_i$ was generated while ending in state l

- **Initialization:**

$$s_{Begin,0} = 1$$

$$s_{k,0} = 0 \text{ for } k \neq Begin.$$

- **Iterate:**

For $i = 1$ to n

For $l = 1$ to $|Q|$

$$s_{li} = e_l(x_i) \cdot \max_{k \in Q} (s_{k,i-1} \cdot a_{kl})$$

- **Output:** $\max s_{k,n}$

Without $\log$

Outcome Likelihood Problem



- **Viterbi algorithm:** s_{li} denote the logarithm of the probability of the most likely path generating the prefix $x_1, \dots, x_i$ and ending in state l
- **Outcome likelihood problem:** o_{li} denote the probability that the prefix $x_1, \dots, x_i$ was generated while ending in state l
- **Initialization:**
 - $o_{Begin,0} = 1$
 - $o_{k,0} = 0$ for $k \neq Begin$.
- **Iterate:**
 - For $i = 1$ to n
 - For $l = 1$ to $|Q|$
 - $$o_{li} = e_l(x_i) \cdot \sum_{k \in Q} (o_{k,i-1} \cdot a_{kl})$$
- **Output:** $\sum_{k \in Q} o_{kn}$

Without $\log$

Outline



1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm – the most probable path
5. Outcome likelihood
- 6. Forward-Backward Algorithm**
7. HMM Parameter Estimation
8. Viterbi training
9. Baum-Welch algorithm
10. Applications of HMM in Biology

Forward-Backward Problem



Given: a sequence $\mathbf{x}$ generated by an HMM M , state k and positive integer i .

Goal: find the probability that M is in a state k when the i -th symbol was emitted.

- Let $P(\mathbf{x}) = \sum_{\pi} P(\mathbf{x}, \pi)$, and
- $P(\mathbf{x}, \pi_i = k) = \sum_{\text{all } \pi \text{ with } \pi_i = k} P(\mathbf{x}, \pi)$,
- then the probability that the HMM was in state k at time i is

$$P(\pi_i = k \mid \mathbf{x}) = \frac{P(\mathbf{x}, \pi_i = k)}{P(\mathbf{x})}$$

Forward Algorithm



- Define $f_{k,i}$ (*forward probability*) as the probability of emitting the prefix $x_1 \cdots x_i$ and reaching the state $\pi_i = k$.
- The recurrence for the forward algorithm:

$$f_{k,i} = e_k(x_i) \cdot \sum_{l \in Q} f_{l,i-1} \cdot a_{lk}$$

How to initialize the computation for

$$f_{l,0} = \begin{cases} 1 & \text{for } l = \text{start state} \\ 0 & \text{for } l \neq \text{start state} \end{cases}$$

Backward Algorithm



- However, *forward probability* is not the only factor affecting $P(\pi_i = k \mid \mathbf{x})$.
- The sequence of transitions and emissions that the HMM undergoes between π_{i+1} and π_n also affect $P(\pi_i = k \mid \mathbf{x})$.

forward x_i backward



Backward Algorithm (cont'd)



- Define *backward probability* $b_{k,i}$ as the probability of being in state $\pi_i = k$ and emitting the *suffix* $x_{i+1} \cdots x_n$.
- The recurrence for the *backward algorithm* :

$$b_{k,i} = \sum_{l \in Q} e_l(x_{i+1}) \cdot b_{l,i+1} \cdot a_{kl}$$

- Initialization:

$$b_{l,n} = a_{l,End}, \text{ for all } l \in Q$$

Backward-Forward Algorithm



- The probability that the model was in state k at step i :

$$P(\pi_i = k \mid \mathbf{x}) = \frac{P(\mathbf{x}, \pi_i = k)}{P(\mathbf{x})} = \frac{f_{k,i} \cdot b_{k,i}}{P(\mathbf{x})}$$

$P(\mathbf{x})$ is the sum of $P(\mathbf{x}, \pi_i = k)$ over all k

Viterbi, Forward, Backward



Viterbi

Initialization:

$$s_{Begin,0} = 1$$

$$s_{k,0} = 0, \text{ for all } k \in Q$$

Iteration:

$$s_{l,i} = e_l(x_i) \cdot \max_{k \in Q} s_{k,i-1} \cdot a_{(k,l)}$$

Termination:

$$P(x, \pi^*) = \max_{k \in Q} s_{k,n}$$

Forward

Initialization:

$$f_{Begin,0} = 1$$

$$f_{k,0} = 0, \text{ for all } k \in Q$$

Iteration:

$$f_{l,i} = e_l(x_i) \cdot \sum_{k \in Q} f_{k,i-1} \cdot a_{k,l}$$

Termination:

$$P(x) = \sum_{k \in Q} f_{k,n} \cdot a_{k,End}$$

Backward

Initialization:

$$b_{k,n} = a_{k,End}, \text{ for all } k \in Q$$

Iteration:

$$b_{l,i} = \sum_{k \in Q} e_k(x_{i+1}) a_{l,k} b_{k,i+1}$$

Termination:

$$P(x) = \sum_{k \in Q} a_{Begin,k} \cdot e_k(x_1) \cdot b_{k,1}$$

Outline



1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm
5. Forward-Backward Algorithm
- 6. HMM Parameter Estimation**
7. Viterbi training
8. Baum-Welch algorithm
9. Applications of HMM in Biology

HMM Parameter Estimation



- So far, we have assumed that the transition and emission probabilities are known.
- However, in most HMM applications, the probabilities are not known. It's very hard to estimate the probabilities.
- Given
 - HMM with **states** and **alphabet** (emission characters)
 - Independent **training sequences** $x^{(1)}, \dots x^{(m)}$ Sequences of different length
- Find HMM parameters Θ (that is, $a_{kl}, e_k(b)$) that **maximize**
$$P(x^{(1)}, \dots x^{(m)} \mid \Theta)$$

the joint probability of the training sequences.

Maximize the likelihood



$P(\mathbf{x}^{(1)}, \dots \mathbf{x}^{(m)} \mid \Theta)$ as a function of Θ is called the **likelihood** of the model.

The training sequences are assumed independent, therefore

$$P(\mathbf{x}^{(1)}, \dots \mathbf{x}^{(m)} \mid \Theta) = \prod_i P(\mathbf{x}^{(i)} \mid \Theta)$$

The parameter estimation problem seeks Θ that realizes

$$\max \prod_i P(\mathbf{x}^{(i)} \mid \Theta)$$

In practice the **log likelihood** is computed to avoid underflow errors

$$\max \sum_i \log P(\mathbf{x}^{(i)} \mid \Theta)$$

Two situations



1. **Known paths** for training sequences
 - CpG islands marked on training sequences
 - One evening the casino dealer allows us to see when he changes dice
2. **Unknown paths**
 - CpG islands are not marked
 - Do not see when the casino dealer changes dice

Known paths



A_{kl} = # times state transition $k \rightarrow l$ occurs in all training sequences

$E_k(b)$ = # times b is emitted from state k in all training sequences

Compute a_{kl} and $e_k(b)$ as maximum likelihood estimators:

$$a_{kl} = \frac{A_{kl}}{\sum_{l'} A_{kl'}}$$

$$e_k(b) = \frac{E_k(b)}{\sum_{b'} E_k(b')}$$

A Parameter Estimations Approach



- If hidden states **were** known, we could use our training data to estimate parameters

$$a_{kl} = \frac{A_{kl}}{\sum_{l'} A_{kl'}}, \quad e_k(b) = \frac{E_k(b)}{\sum_{b'} E_k(b')}$$

- However, usually the hidden state sequence π is not given, but only the observed output stream x
- An alternative is to make an intelligent guess of π , use the equations above to estimate parameters, then run Viterbi to estimate the hidden state, then re-estimate the parameters and repeat until the state assignments or parameter values converge.
- Such iterative approaches are called Expectation Maximization (EM) methods of parameter estimation

Pseudocounts



- Some state k may not appear in any of the training sequences. This means $A_{kl} = 0$ for every state l and a_{kl} cannot be computed with the given equation.
- To avoid this **overfitting** use predetermined **pseudocounts** r_{kl} and $r_k(b)$.

$$A_{kl} = \# \text{ of transitions } k \rightarrow l + r_{kl}$$

$$E_k(b) = \# \text{ of emissions of } b \text{ from } k + r_k(b)$$

The pseudocounts can reflect our prior biases about the probability values.

Outline



1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm
5. Forward-Backward Algorithm
6. HMM Parameter Estimation
7. **Viterbi training**
8. Baum-Welch algorithm
9. Applications of HMM in Biology

Unknown paths: Viterbi training



Idea: use Viterbi decoding to compute the most probable path for training sequence x .

Start with some guess for initial parameters and compute π^* the most probable path for x using initial parameters.

Iterate until no change in π^* :

- Determine A_{kl} and $E_k(b)$ as before

- Compute new parameters a_{kl} and $e_k(b)$ using the same formulas as before

- Compute new π^* for x and the current parameters

Viterbi training analysis



- The algorithm **converges precisely**.
 - There are finitely many possible paths.
 - New parameters are uniquely determined by the current π^* .
 - There may be several paths for x with the same probability, hence (when testing whether π^* changed) must compare the new π^* with all previous paths having highest probability.
- Does **not maximize the likelihood** $\prod_x P(x | \Theta)$ but the contribution to the likelihood of the most probable path $\prod_x P(x | \Theta, \pi^*)$
- In general, performs less well than Baum-Welch

Outline



1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm
5. Forward-Backward Algorithm
6. HMM Parameter Estimation
7. Viterbi training
8. **Baum-Welch algorithm**
9. Applications of HMM in Biology

Unknown paths: Baum-Welch



Idea:

1. Guess initial values for parameters.
art and experience, not science
2. Estimate new (better) values for parameters.
how?
3. Repeat until stopping criteria is met.
what criteria?

Better values for parameters



- Would need the A_{kl} and $E_k(b)$ values but cannot count (the path is unknown) and do not want to use the most probable path.
- For all states k, l , symbol b and training sequence x

Compute A_{kl} and $E_k(b)$ as **expected values**, given the current parameters

Notation:

- For any sequence of characters x emitted along some unknown path π , denote by $\pi_i = k$ the assumption that the state at position i (in which x_i is emitted) is k .

Probabilistic setting for A_{kl}



- Given $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$ consider a **discrete probability space** with **elementary events**

$$\varepsilon_{k,l} = "k \rightarrow l \text{ is taken in } \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}"$$

- For each $\mathbf{x}$ in $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ and each position i in $\mathbf{x}$ let $Y_{x,i}$ be a **random variable** defined by

$$Y_{x,i}(\varepsilon_{k,l}) = \begin{cases} 1 & \text{if } \pi_i = k \text{ and } \pi_{i+1} = l \text{ in } \mathbf{x} \\ 0 & \text{otherwise} \end{cases}$$

- Define $Y = \sum_{\mathbf{x}} \sum_i Y_{x,i}$ random variable that counts # of times the event $\varepsilon_{k,l}$ happens in $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$.

The meaning of A_{kl}



Let A_{kl} be the expectation of Y

$$\begin{aligned} E(Y) &= \sum_x \sum_i E(Y_{x,i}) = \sum_x \sum_i P(Y_{x,i} = 1) = \\ &= \sum_x \sum_i P(\{\varepsilon_{k,l} ; \pi_i = k \text{ and } \pi_{i+1} = l \text{ in } \mathbf{x}\}) = \\ &= \sum_x \sum_i P(\pi_i = k \text{ and } \pi_{i+1} = l \mid \mathbf{x}) \end{aligned}$$

Need to compute $P(\pi_i = k \text{ and } \pi_{i+1} = l \mid \mathbf{x})$

Probabilistic setting for $E_k(b)$



Given $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$ consider a **discrete probability space** with **elementary events**

$$\varepsilon_{k,b} = "b \text{ is emitted in state } k \text{ in } \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}"$$

For each $\mathbf{x}$ in $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ and each position i in $\mathbf{x}$ let $Y_{x,i}$ be a **random variable** defined by

$$Y_{x,i}(\varepsilon_{k,b}) = \begin{cases} 1 & \text{if } x_i = b \text{ and } \pi_i = k \text{ in } \mathbf{x} \\ 0 & \text{otherwise} \end{cases}$$

Define $Y = \sum_{\mathbf{x}} \sum_i Y_{x,i}$ random variable that counts # of times the event $\varepsilon_{k,b}$ happens in $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$.

The meaning of $E_k(b)$



Let $E_k(b)$ be the expectation of Y

$$\begin{aligned} E(Y) &= \sum_x \sum_i E(Y_{x,i}) = \sum_x \sum_i P(Y_{x,i} = 1) = \\ &= \sum_x \sum_i P(\{\varepsilon_{k,b} ; x_i = b \text{ and } \pi_i = k \text{ in } \mathbf{x}\}) = \\ &= \sum_x \sum_{\{i \mid x_i = b\}} P(\{\varepsilon_{k,b} ; x_i = b, \pi_i = k\}) = \sum_x \sum_{\{i \mid x_i = b\}} P(\{\pi_i = k \mid \mathbf{x}\}) \end{aligned}$$

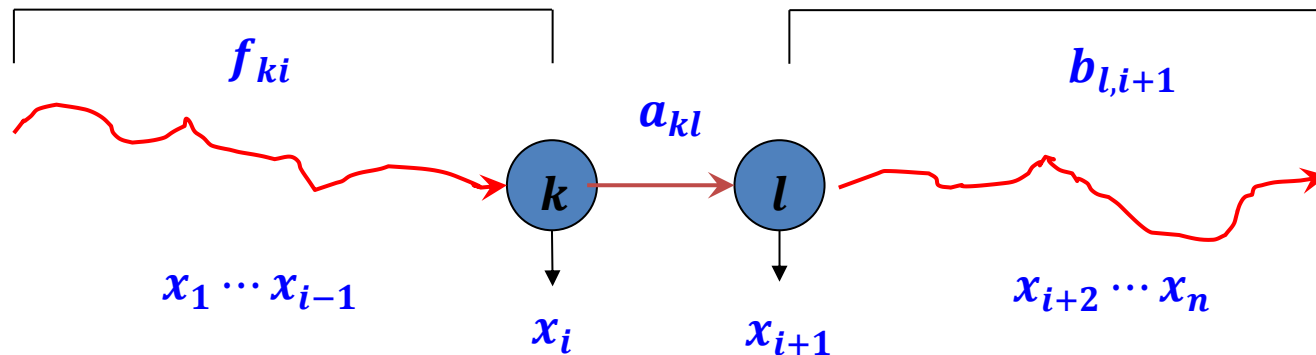
Need to compute $P(\pi_i = k \mid \mathbf{x})$

Computing new parameters



Consider $\mathbf{x} = x_1 \cdots x_n$ training sequence

Concentrate on positions i and $i + 1$



Use the forward-backward values:

$$f_{ki} = P(x_1 \cdots x_i, \pi_i = k)$$

$$b_{ki} = P(x_{i+1} \cdots x_n \mid \pi_i = k)$$

Compute A_{kl} (1)



- Prob $k \rightarrow l$ is taken at position i of $\mathbf{x}$
$$P(\pi_i = k, \pi_{i+1} = l \mid x_1 \cdots x_n) = P(\mathbf{x}, \pi_i = k, \pi_{i+1} = l) / P(\mathbf{x})$$
- Compute $P(\mathbf{x})$ using either forward or backward values
- We'll show that
$$P(\mathbf{x}, \pi_i = k, \pi_{i+1} = l) = b_{l,i+1} \cdot e_l(x_{i+1}) \cdot a_{kl} \cdot f_{ki}$$
- Expected # times $k \rightarrow l$ is used in training sequences
$$A_{kl} = \sum_{\mathbf{x}} \sum_i (b_{l,i+1} \cdot e_l(x_{i+1}) \cdot a_{kl} \cdot f_{ki}) / P(\mathbf{x})$$

Compute A_{kl} (2)



$$P(\mathbf{x}, \pi_i = k, \pi_{i+1} = l) =$$

$$P(x_1 \cdots x_i, \pi_i = k, \pi_{i+1} = l, x_{i+1} \cdots x_n) =$$

$$P(A, B) = P(A | B) \cdot P(B)$$

$$P(\pi_{i+1} = l, x_{i+1} \cdots x_n \mid x_1 \cdots x_i, \pi_i = k) \cdot P(x_1 \cdots x_i, \pi_i = k) =$$

$$P(\pi_{i+1} = l, x_{i+1} \cdots x_n \mid \pi_i = k) \cdot f_{ki} =$$

$$P(x_{i+1} \cdots x_n \mid \pi_i = k, \pi_{i+1} = l) \cdot P(\pi_{i+1} = l \mid \pi_i = k) \cdot f_{ki} =$$

$$P(x_{i+1} \cdots x_n \mid \pi_{i+1} = l) \cdot a_{kl} \cdot f_{ki} =$$

$$P(x_{i+2} \cdots x_n \mid x_{i+1}, \pi_{i+1} = l) \cdot P(x_{i+1} \mid \pi_{i+1} = l) \cdot a_{kl} \cdot f_{ki} =$$

$$P(x_{i+2} \cdots x_n \mid \pi_{i+1} = l) \cdot e_l(x_{i+1}) \cdot a_{kl} \cdot f_{ki} =$$

$$b_{l,i+1} \cdot e_l(x_{i+1}) \cdot a_{kl} \cdot f_{ki}$$

Compute $E_k(b)$



Probability x_i of $\mathbf{x}$ is emitted in state k

$$P(\pi_i = k \mid x_1 \dots x_n) = P(\pi_i = k, x_1 \dots x_n) / P(\mathbf{x})$$

$$P(\pi_i = k, x_1 \dots x_n) = P(x_1 \dots x_i, \pi_i = k, x_{i+1} \dots x_n) =$$

$$P(x_{i+1} \dots x_n \mid x_1 \dots x_i, \pi_i = k) \cdot P(x_1 \dots x_i, \pi_i = k) =$$

$$P(x_{i+1} \dots x_n \mid \pi_i = k) \cdot f_{ki} = b_{ki} \cdot f_{ki}$$

Expected # times b is emitted in state k

$$E_k(b) = \sum_{\mathbf{x}} \frac{\sum_{i: x_i=b} (f_{ki} \cdot b_{ki})}{P(\mathbf{x})} = \sum_{\mathbf{x}} \sum_{i: x_i=b} \frac{f_{ki} \cdot b_{ki}}{P(\mathbf{x})}$$

Finally, new parameters



Can add pseudocounts as before.

$$a_{kl} = \frac{A_{kl}}{\sum_{l'} A_{kl'}}$$

$$e_k(b) = \frac{E_k(b)}{\sum_{b'} E_k(b')}$$

Stopping criteria



We cannot actually reach the maximum (optimization of continuous functions).

Therefore, there is a need for stopping criteria.

- Compute the **log**-likelihood of the model for the current Θ

$$\sum_x \log P(x | \Theta)$$

- Compare with previous **log** likelihood.
- Stop if small difference.
- Stop after a certain number of iterations.

The Baum-Welch algorithm



Initialization:

Pick the best-guess for model parameters
(or arbitrary)

Iteration:

1. Forward for each x
2. Backward for each x
3. Calculate A_{kl} , $E_k(b)$
4. Calculate new a_{kl} , $e_k(b)$
5. Calculate new log-likelihood

Until log-likelihood does not change much

Baum-Welch analysis



- Log-likelihood is increased by iterations
Baum-Welch is a particular case of the EM (expectation maximization) algorithm
- Convergence to local maximum. Choice of initial parameters determines local maximum to which the algorithm converges

Implementation Issue – Scaling



- To compute $f_k(i)$ and $b_k(i)$, multiplication of a large number of terms (probability), value heads to 0 quickly, which exceed the precision range of any machine.
- The basic procedure is to multiply them by a scaling coefficient that is independent of i (i.e., it depends only on k). Logarithm cannot be used because of summation. But we can use

$$c_t = \frac{1}{\sum_{k=1}^n f_k(i)}$$

- c_t will be stored for the time points when the scaling is performed. c_t is used for both $f_k(i)$ and $b_k(i)$. The scaling factor will be canceled out for parameter estimation.
- For Viterbi algorithm, the use of logarithm is O.K.

Outline



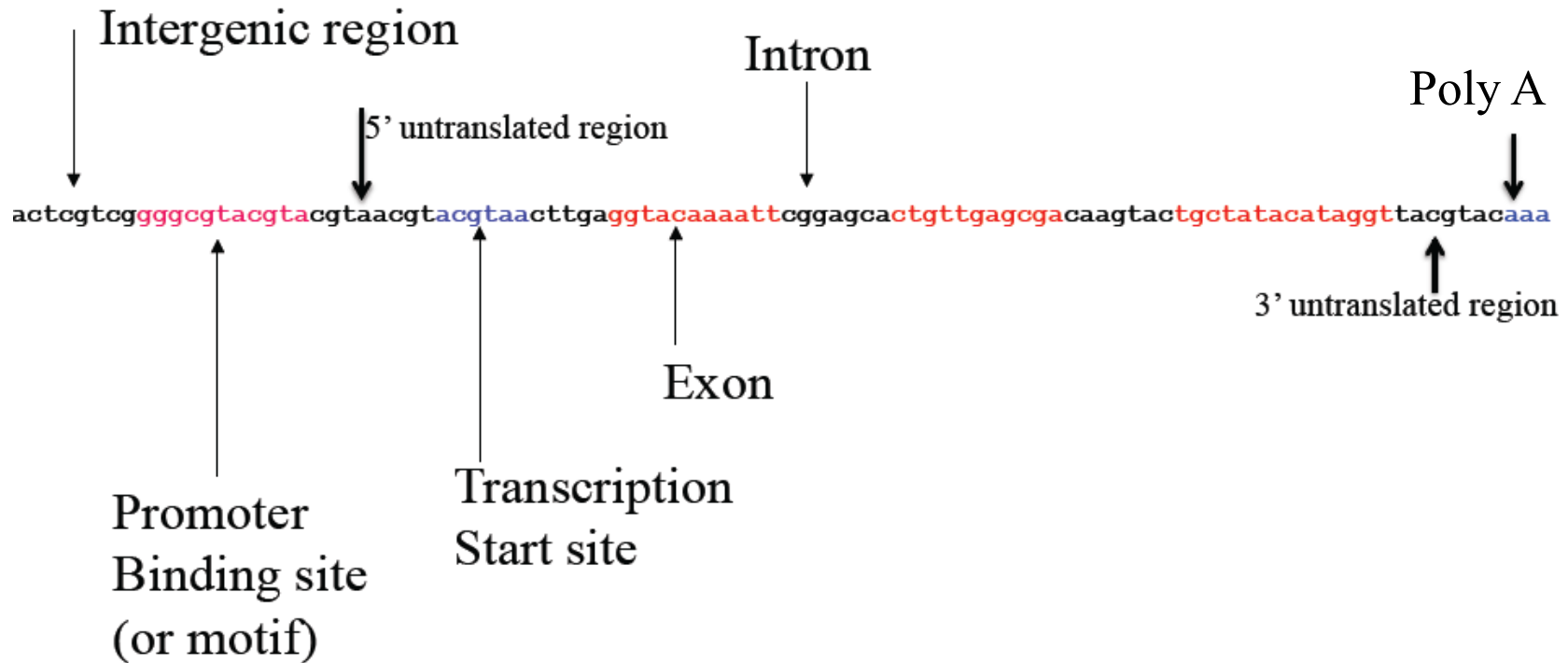
1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm
5. Forward-Backward Algorithm
6. HMM Parameter Estimation
7. Viterbi training
8. Baum-Welch algorithm
9. **Applications of HMM in Biology**
 - A. Finding genes
 - B. Profile HMM
 - C. Pairwise Alignment via HMM

Application of HMM in Biological Sequence Analysis



- Gene prediction
- Protein sequence modeling (learning, profile)
- Protein sequence alignment (decoding)
- Protein database search (scoring, e.g., fold recognition)
- Protein structure prediction
- ...

Motif and Gene Structure



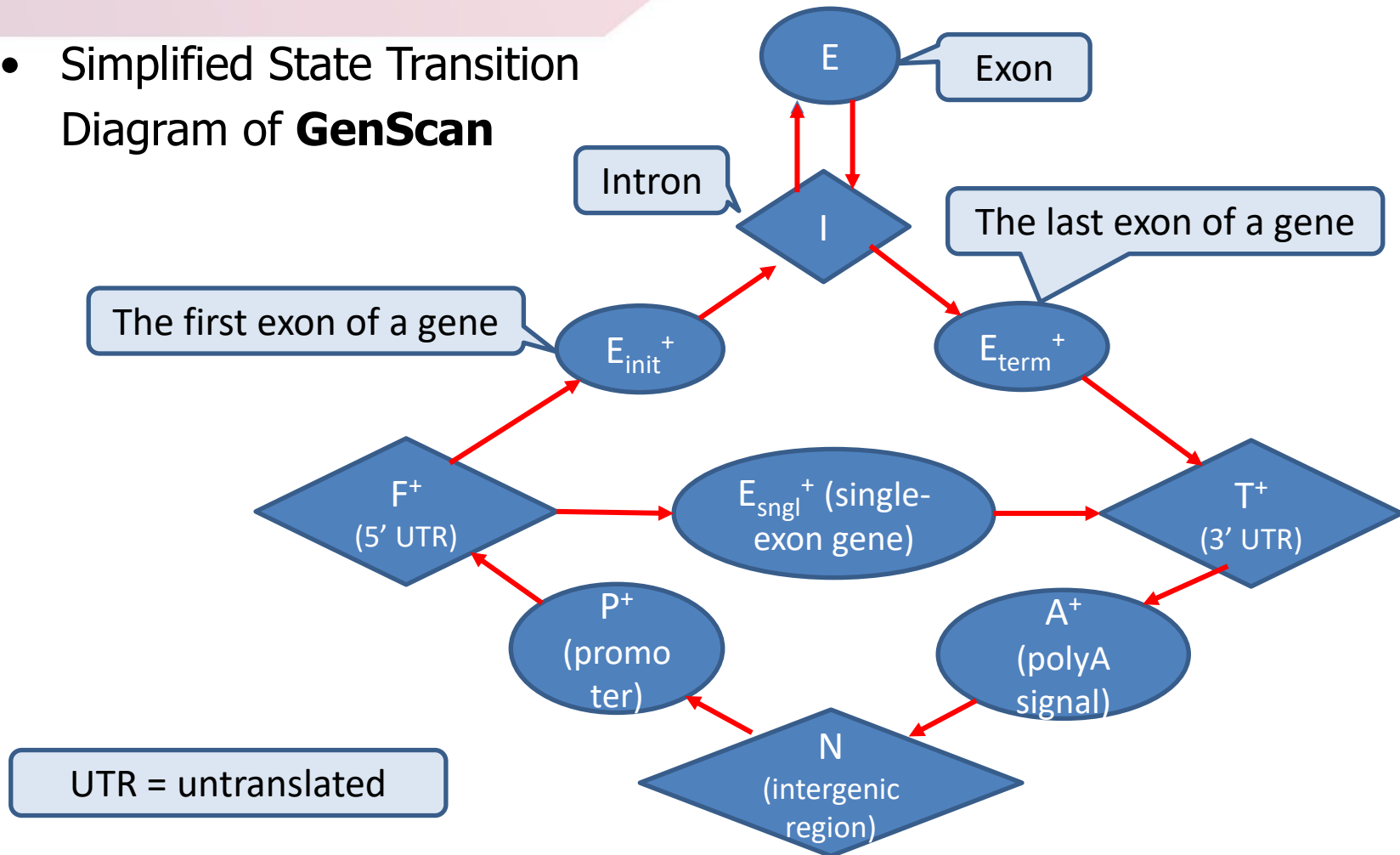
- HMM has been used for modeling binding site and gene structure prediction.

GENSCAN

(<https://web.archive.org/web/20130906115338/http://genes.mit.edu/GENSCAN.html>)



- Simplified State Transition Diagram of **GenScan**



Outline



1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm
5. Forward-Backward Algorithm
6. HMM Parameter Estimation
7. Viterbi training
8. Baum-Welch algorithm
9. Applications of HMM in Biology
 - A. Finding genes
 - B. Profile HMM**
 - C. Pairwise Alignment via HMM

Finding Distant Members of a Protein Family



- A distant cousin of functionally related sequences in a protein family may have weak pairwise similarities with each member of the family and thus fail significance test.
- However, they may have weak similarities with *many* members of the family.
- The goal is to align a sequence to *all* members of the family at once.
 - However multiple alignment is computationally expensive
- **A solution:** A family of related proteins can be represented by their multiple alignment and the corresponding profile.

Profile Representation of Protein Families



- Aligned DNA sequences (without gaps) can be represented by a $4 \times n$ profile matrix reflecting the frequencies of nucleotides in every aligned position.

A	.72	.14	0	0	.72	.72	0	0
T	.14	.72	0	0	0	.14	.14	.86
G	.14	.14	.86	.44	0	.14	0	0
C	0	0	.14	.56	.28	0	.86	.14

- Protein family can be represented by a $20 \times n$ profile representing frequencies of amino acids, **but**
 - an alignment can contain gaps and insertions
 - a profile does not preserve information about consecutive bases

Why do We Build a Profile (Model)?



- Understand the conservation (core function and structure elements) and variation
- Sequence generation
- Multiple sequence alignments
- Profile-sequence alignment (more sensitive than sequence-sequence alignment)
- Family/fold recognition
- Profile-profile alignment

Profiles and HMMs



- HMMs can be used for aligning a sequence against a profile representing a protein family.
- A $20 \times n$ profile P corresponds to n sequentially linked *match* states $M_1, \dots, M_n$ in the **profile HMM** of P .
- Multiple alignment of a protein family shows variations in conservation along the length of a protein
- Example: after aligning many globin proteins, the biologists recognized that the helical regions in globins are more conserved than others.

What are Profile HMMs?



- A Profile HMM is a probabilistic representation of a multiple alignment.
- A given multiple alignment (of a protein family) is used to build a profile HMM.
- This model may then be used to find and score less obvious potential matches of new protein sequences.

Sequence alignment (for those who do not know it)



- Two sequences are aligned by writing them in two rows. Identical or similar characters are placed in the same column, whereas non-identical characters are either placed in the same column as a mismatch or are opposite a gap in the other sequence.

Two strings:



Alignment:

IMISSMISSISSIPPI

I-MISSMISSISSIPPI-

||| ||| ||| ||| ||| ||| ||| ||| ||| |||

MYMISSISAHIPPIE

MYMISS-ISAH--IPPIE

Different lengths

The same lengths

Similarity Score



- For $\Sigma = \{A, B, L, -\}$ consider the following similarity score matrix S :

	A	B	L	-
A	3	1	-1	-2
B		2	0	-3
L			1	2
-				0

- matches of symbols are rewarded, mismatches and gaps penalized.

$X = B \quad L \quad A \quad - \quad B \quad L \quad A$

$Y = A \quad L \quad A \quad B \quad B \quad L \quad -$

$$1 + 1 + 3 - 3 + 2 + 1 - 2 = 3$$

Global Alignment



Input: Strings x and y and a scoring schema

Output: Alignment of maximum score

$\downarrow \rightarrow = -\sigma$

$\searrow \begin{cases} = 1 & \text{if match} \\ = -\mu & \text{if mismatch} \end{cases}$

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + 1 & \text{if } x_i = y_j \\ F(i-1, j-1) - \mu & \text{if } x_i \neq y_j \\ F(i-1, j) - \sigma \\ F(i, j-1) - \sigma \end{cases}$$

	0	y_1	$\cdots$	y_{j-1}		y_j	$\cdots$	y_m
0	0	1	$\cdots$	$j-1$		j	$\cdots$	m
x_1	1							
$\vdots$	$\vdots$							
x_{i-1}	$i-1$			$F(i-1, j-1)$		$F(i-1, j)$		
					$\ddots$	$\downarrow$		
x_i				$F(i, j-1)$	$\rightarrow$	$F(i, j)$		
$\vdots$								
x_n								

Gap Penalties



- Gaps are undesirable and thus penalized. The standard cost associated with a gap of length g is given either by a **linear score**

$$\gamma(g) = -gd$$

or an **affine score**

$$\gamma(g) = -d - (g - 1)e$$

where d is the gap open penalty and e is the gap extension penalty.

- Usually, $e < d$, with the result that **lower number of** isolated gaps are produced, as shown in the following comparison:

Linear gap penalty: GSAQVKGHGKKVADALTNAVAHVDDMPNALSALSDLHAHKL
GSAQVKGHGKK-----VA--D----A-SALSDLHAHKL

Affine gap penalty: GSAQVKGHGKKVADALTNAVAHVDDMPNALSALSDLHAHKL
GSAQVKGHGKKVADA-----SALSDLHAHKL

Affine-Gap Algorithm



- Instead of using one matrix $F(i, j)$ to represent the best score attainable up to x_i and y_i , we will now use three matrices M , I_x and I_y :
 - $M(i, j)$ is the best score up to (i, j) , given that x_i is aligned to y_j ,
 - $I_x(i, j)$ is the best score up to (i, j) , given that y_j is aligned to a gap, and
 - $I_y(i, j)$ is the best score up to (i, j) , given that x_i is aligned to a gap in y .
- **Initialization:**
 $M(0, 0) = 0$, $I_x(0, 0) = I_y(0, 0) = -\infty$
 $I_x(0, j) = -d - (j - 1)e$, $M(0, j) = I_y(0, j) = -\infty$, for $j = 1, \dots, m$, and
 $I_y(i, 0) = -d - (i - 1)e$, $M(i, 0) = I_x(i, 0) = -\infty$, for $i = 1, \dots, n$.
- We assume that a gap in one sequence is not immediately followed by a gap in the other. This is true for the optimal path, if $-d - e$ is less than the lowest mismatch score.

Affine Gap Global Alignment



- $M(i, j)$ is the best score up to (i, j) , given that x_i is aligned to y_j ,
- $I_x(i, j)$ is the best score up to (i, j) , given that y_j is aligned to a gap, and
- $I_y(i, j)$ is the best score up to (i, j) , given that x_i is aligned to a gap in y .
- **Recursion:**

$$I_x(i, j) = \max \begin{cases} M(i, j-1) - d & \text{begin gap in } X \\ I_x(i, j-1) - e & \text{continue gap in } X \end{cases}$$

$$I_y(i, j) = \max \begin{cases} M(i-1, j) - d & \text{begin gap in } Y \\ I_y(i-1, j) - e & \text{continue gap in } Y \end{cases}$$

$$M(i, j) = \max \begin{cases} M(i-1, j-1) + s(x_i, y_j) & \text{match or mismatch} \\ I_x(i-1, j-1) + s(x_i, y_j) & \text{end gap in } X \\ I_y(i-1, j-1) + s(x_i, y_j) & \text{end gap in } Y \end{cases}$$

Multiple Sequence Alignment



- Based on a score matrix, sequences are aligned with gaps

	A	B	C	D	...	-
A	5	1	-2	-1	...	-5
B	1	6	4	-3	...	-5
C	-2	4	5	-4	...	-5
D	-1	-3	-4	4	...	-5

gap

- Unaligned sequences

```
AABNFCAQCDTYBNNBBTYANGC
AACFCBNFQADNNBCDTYBNANBAGC
```

- Alignment with **gaps** (indels) and **mismatches**

```
AABNFCA--QCDTYBNNBB--TY--AN--GC
AAC--FCBNFQAD---NNBCDTYBNANBAGC
```

Key to Build an HMM is to Set Up States

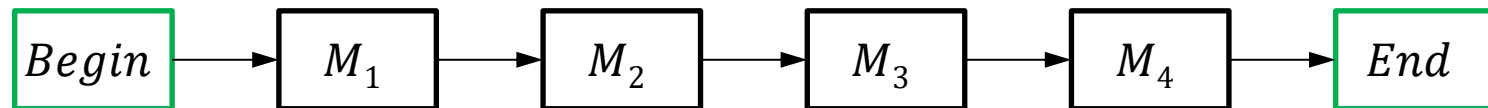


- Think about the positions of the ancestral sequence that is undergoing mutation events to generate new sequences in different species. A position can be modeled by a **dice**.
1. **Match** (match or mutate): the position is kept with or without variations/mutations.
 2. **Delete**: the position is deleted.
 3. **Insert**: amino acids are inserted between two positions.

Hidden Markov Model



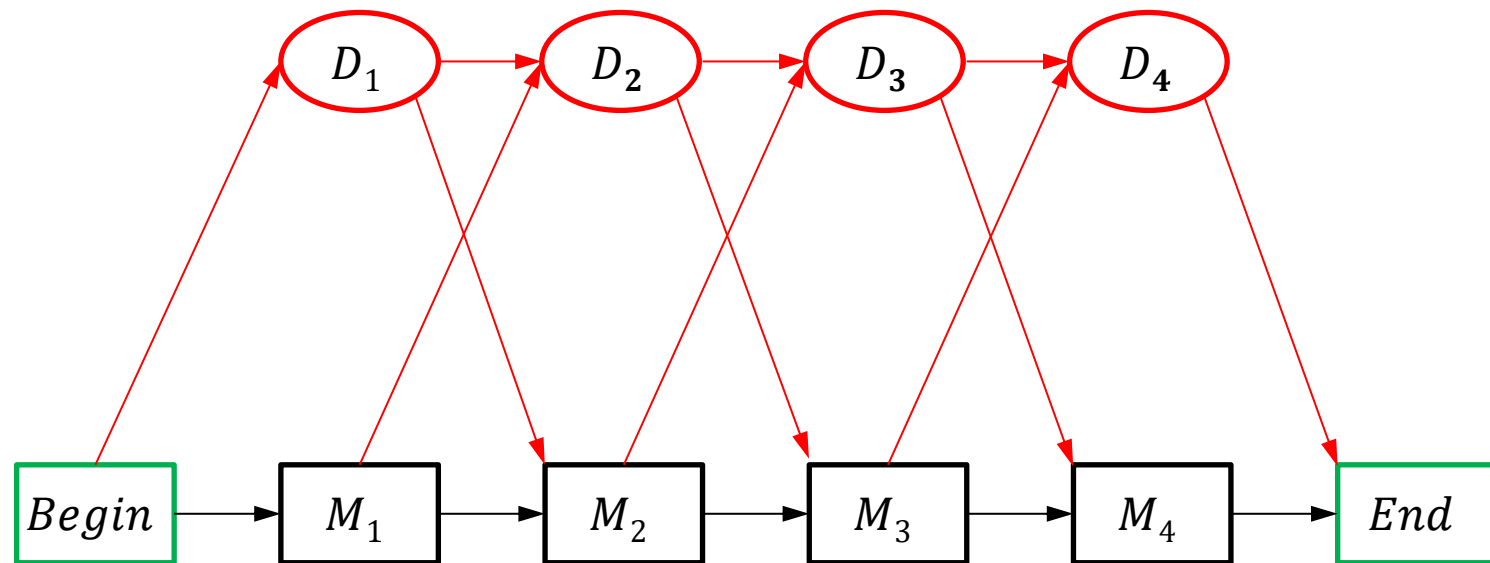
- Each match state has an emission distribution of 20 amino acids; one match state for a position.



Hidden Markov Model



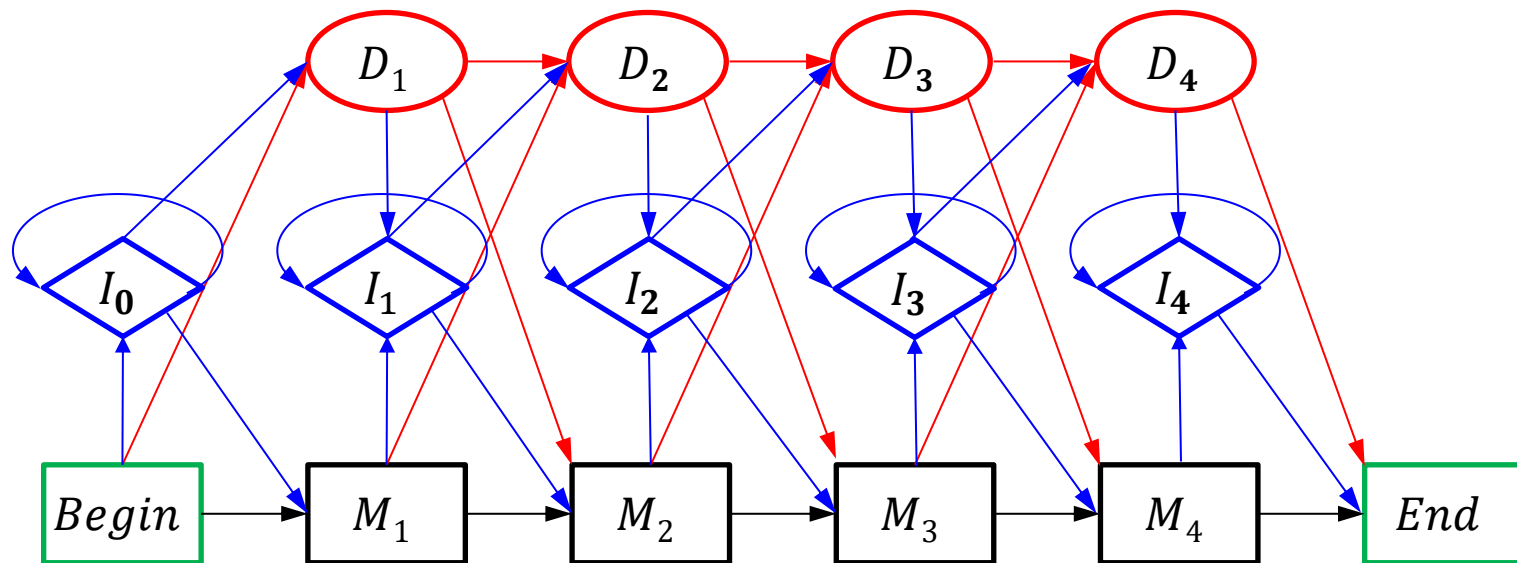
- Each match state has an emission distribution of 20 amino acids; one match state for a position.
- Deletion state is a mute state (emitting a dummy)



Hidden Markov Model



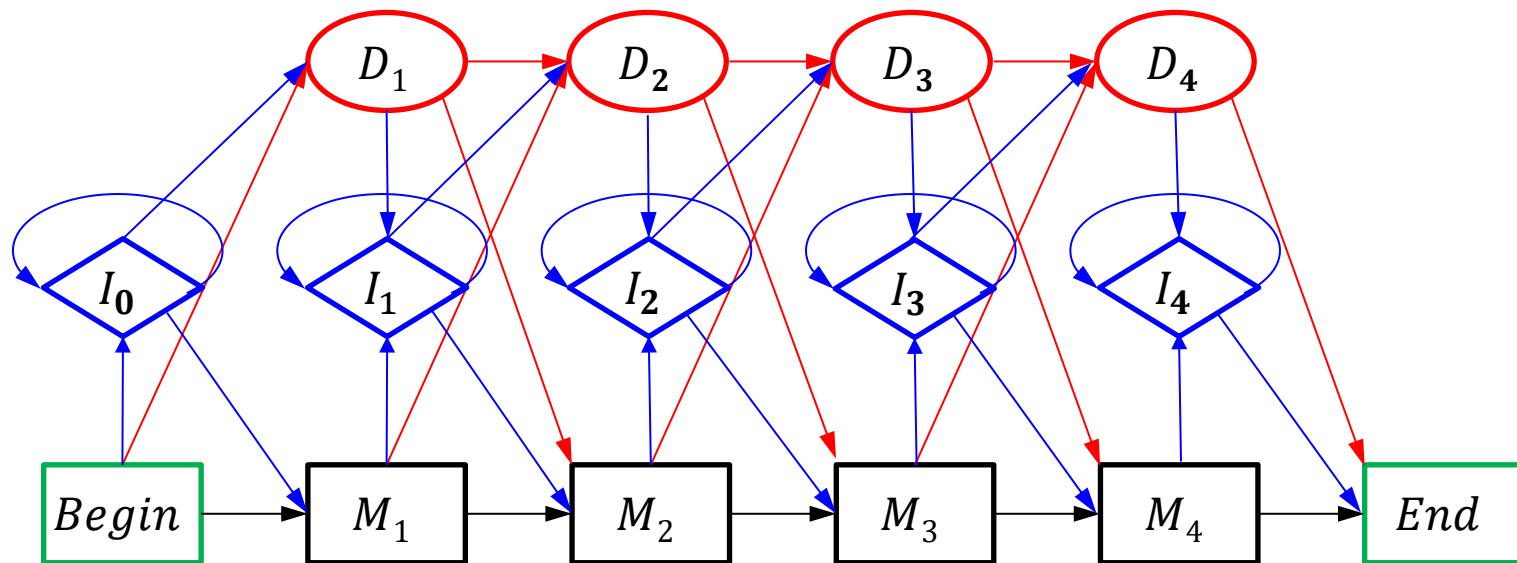
- Each match state has an emission distribution of 20 amino acids; one match state for a position.
- Deletion state is a mute state (emitting a dummy)
- Each insertion state has an emission distribution of 20 amino acids.



Hidden Markov Model



- How many **states**? (n positions: length of model)
 $n(\text{match}) + n(\text{deletion}) + (n + 1)(\text{insertion}) + 2 = 3n + 3$

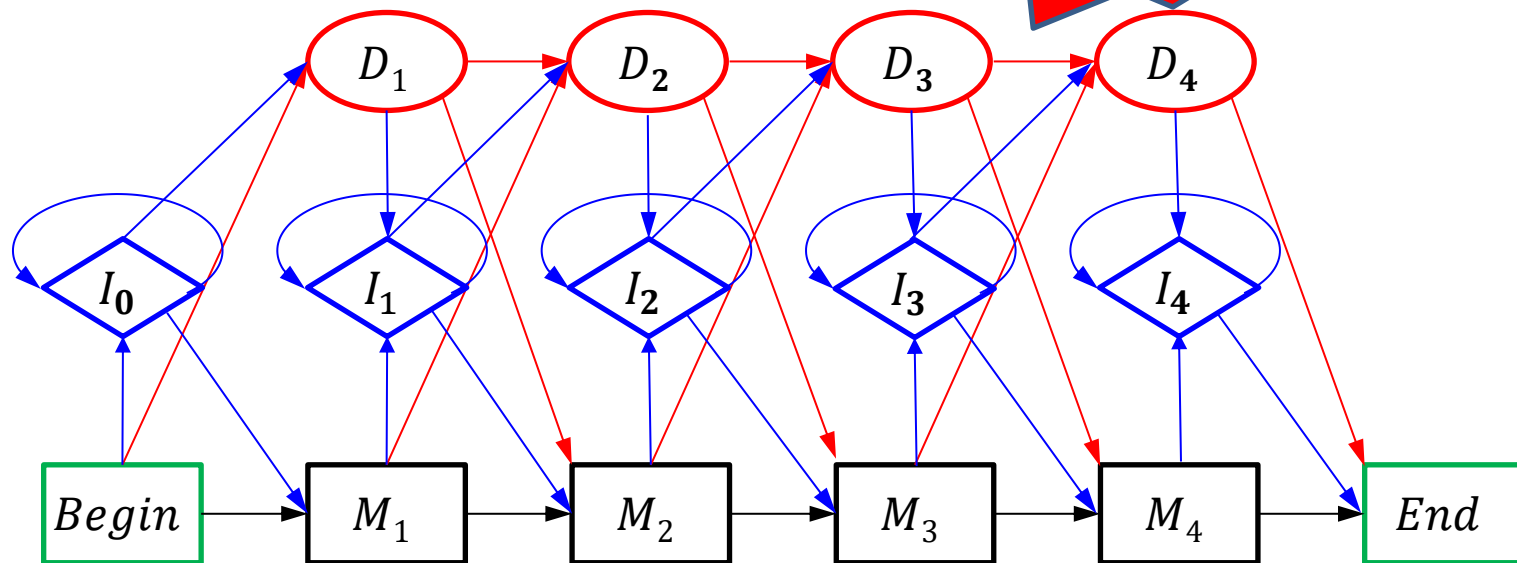


Hidden Markov Model



- How many **transitions**? (n positions = length of the model)
- Deletion: $3n - 1$, Match: $3n - 1$, Insertion: $3(n + 1) - 1$, B/E: 3
- Total = $9n + 3$.

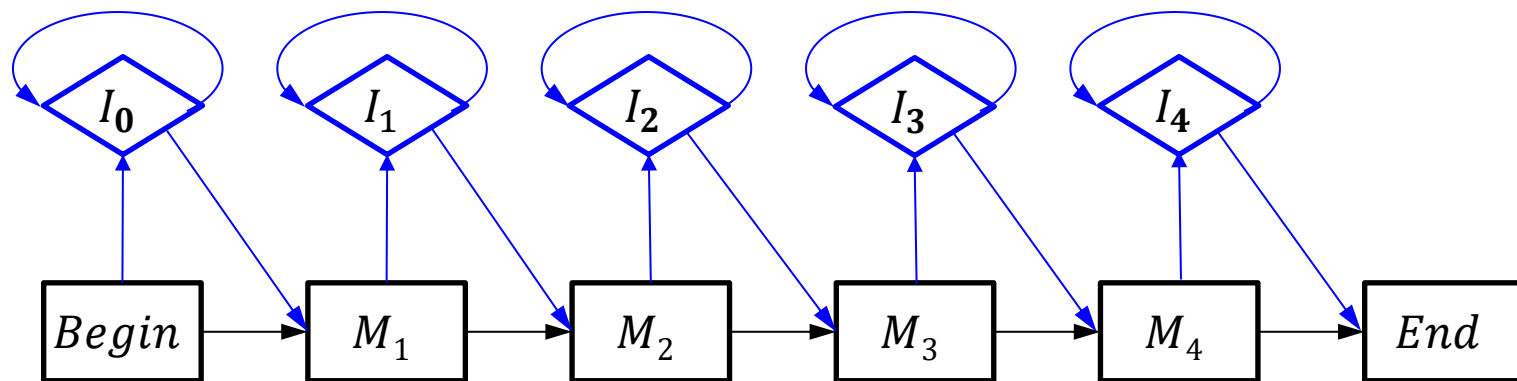
Why deletion nodes?



Hidden Markov Model



- How many **transitions**? (n positions = length of the model)
- Deletion: $3n - 1$, Match: $3n - 1$, Insertion: $3(n + 1) - 1$, B/E: 3
- Total = $9n + 3$.
- How many **emission probabilities**?



Initialization of HMM



- **How to decide model length (the number of match states)?**
 - **Learn:** Use a range of model lengths (centered at the average sequence length). If transition probability from a match (M_i) state to a delete state (D_{i+1}) > 0.5 , remove the M_{i+1} . If transition probability from a match (M_i) state to an insertion state (I_{i+1}) > 0.5 , add a match state.
 - **Get from multiple alignment:** assign a match state to any column with $< 50\%$ gaps.
- How to initialize transition probabilities?
- How to initialize emission probabilities?

Initialization of HMM



- **How to decide model length (the number of match states)?**
- **How to initialize transition probabilities?**
 - Uniform initialization of transition probabilities is O.K. in most cases.
- **How to initialize emission probabilities?**
 - Uniform initialization of emission probability of insert state is O.K. in many cases.
 - Uniform initialization of emission probability of match state is bad. (leads to **bad** local minima)
 - Using amino acid distribution to initialize the emission probabilities is better. (need regularization / smoothing to avoid zero)

Initialize from Multiple Alignments



```
seq1 ---VRRNNMGPLIESSSYHDALFTLGY--AGDRISQMLGMRLLAQGRLSEMAGADALDV
seq2 --NIYIDSNGLAHYANNLHDLFLAEGY--EASQRLFEIELFG-LAMGNLSSWVGAKALSS
seq3 SAETYRDAWGIPHLRADTPHELARAQGT--ARDRAWQLEVERHRAQGTSASFLGPEALSW
seq4 -----DRLGVVTIDAANQLDAMRALGY-AQERYFEMDLMRRAPAGELSELFgakAVDL
```

- First, assign match/main states, delete states, insert states from multiple sequence alignment
- Get the path of each sequence
- Count the amino acid frequencies emitted from match or insert states, which are converted into probabilities for each state (need smoothing/regularization/pseudo-count).
- Count the number of state transitions and use them to initialize transition probabilities.

Profile HMM from Multiple Sequence Alignment



- Less than half gaps in columns 1, 2, 6
 - Columns 1,2,6 are match states M_1, M_2, M_3
- Columns 3,4,5 more than half gaps
 - Create a single insert state I_2
- Emission probabilities
 - $e_{M_1}(B) = \frac{3}{3}$ $e_{M_1}(A) = \frac{0}{3}$ $e_{M_1}(T) = \frac{0}{3}$...
- Zero probabilities cause problems (overfitting) – use Laplace correction (add 1; pseudo-counts)
 - $e_{M_1}(B) = \frac{3+1}{3+20}$ $e_{M_1}(A) = \frac{0+1}{3+20}$ $e_{M_1}(T) = \frac{0+1}{3+20}$...
- What are the emission probabilities $e_{M_2}(.)$, $e_{M_3}(.)$?

M_1	M_2	I_2			M_3
B	A	–	–	–	Q
B	A	G	–	C	Q
–	T	G	–	–	Q
B	T	–	F	–	Q
–	A	–	–	C	–
1	2	3	4	5	6

Profile HMM from Multiple Sequence Alignment



- Emission probabilities

- $$e_{I_2}(G) = \frac{2}{5} \quad e_{I_2}(A) = \frac{0}{5} \quad e_{I_2}(F) = \frac{1}{5} \dots$$

- zero probabilities cause problems (overfitting) – use Laplace correction (add 1; pseudo-counts)

- $$e_{I_2}(G) = \frac{2+1}{5+20} \quad e_{I_2}(A) = \frac{0+1}{5+20} \quad e_{I_2}(F) = \frac{1+1}{5+20} \dots$$

- Transition probabilities

- $$a_{Begin, M_1} = \frac{3}{5} \quad a_{Begin, D_1} = \frac{2}{5} \quad a_{Begin, I_0} = \frac{0}{5} \dots$$

- zero probabilities cause problems (overfitting) – use Laplace correction (add 1; pseudo-counts)

- $$a_{Begin, M_1} = \frac{3+1}{5+3} \quad a_{Begin, D_1} = \frac{2+1}{5+3}$$

$$a_{Begin, I_0} = \frac{0+1}{5+3} \dots$$

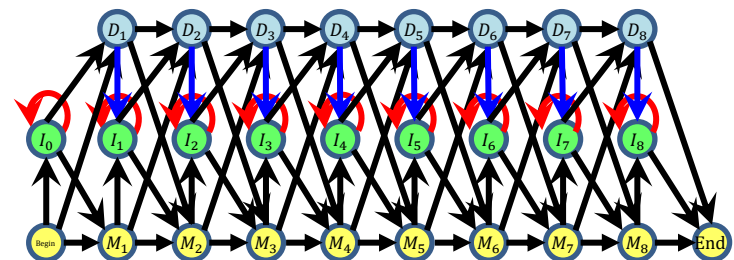
M_1	M_2	I_2			M_3
B	A	–	–	–	Q
B	A	G	–	C	Q
–	T	G	–	–	Q
B	T	–	F	–	Q
–	A	–	–	C	–
1	2	3	4	5	6

Profile HMM from Multiple Sequence Alignment



- When there is no information in the alignment – set the probabilities to uniform
- I_1 does not appear in the alignment
 - $a_{I_1, M_2} = a_{I_1, I_1} = a_{I_1, D_2} = \frac{1}{3}$
- Transition from the delete state D_1 only into M_2
 - $a_{D_1, M_2} = \frac{5}{5} \quad a_{D_1, I_1} = \frac{0}{5} \quad a_{D_1, D_2} = \frac{0}{5}$
- Again add 1 to the counts
 - $a_{D_1, M_2} = \frac{5+1}{5+3} \quad a_{D_1, I_1} = \frac{0+1}{5+3} \quad a_{D_1, D_2} = \frac{0+1}{5+3}$
- What are the emission probabilities
 - $e_{D_1}(A) = ? \quad e_{D_1}(B) = ? \quad \dots$

M_1	M_2	I_2			M_3
B	A	–	–	–	Q
B	A	G	–	C	Q
–	T	G	–	–	Q
B	T	–	F	–	Q
–	A	–	–	C	–
1	2	3	4	5	6



Probabilities in Profile HMM



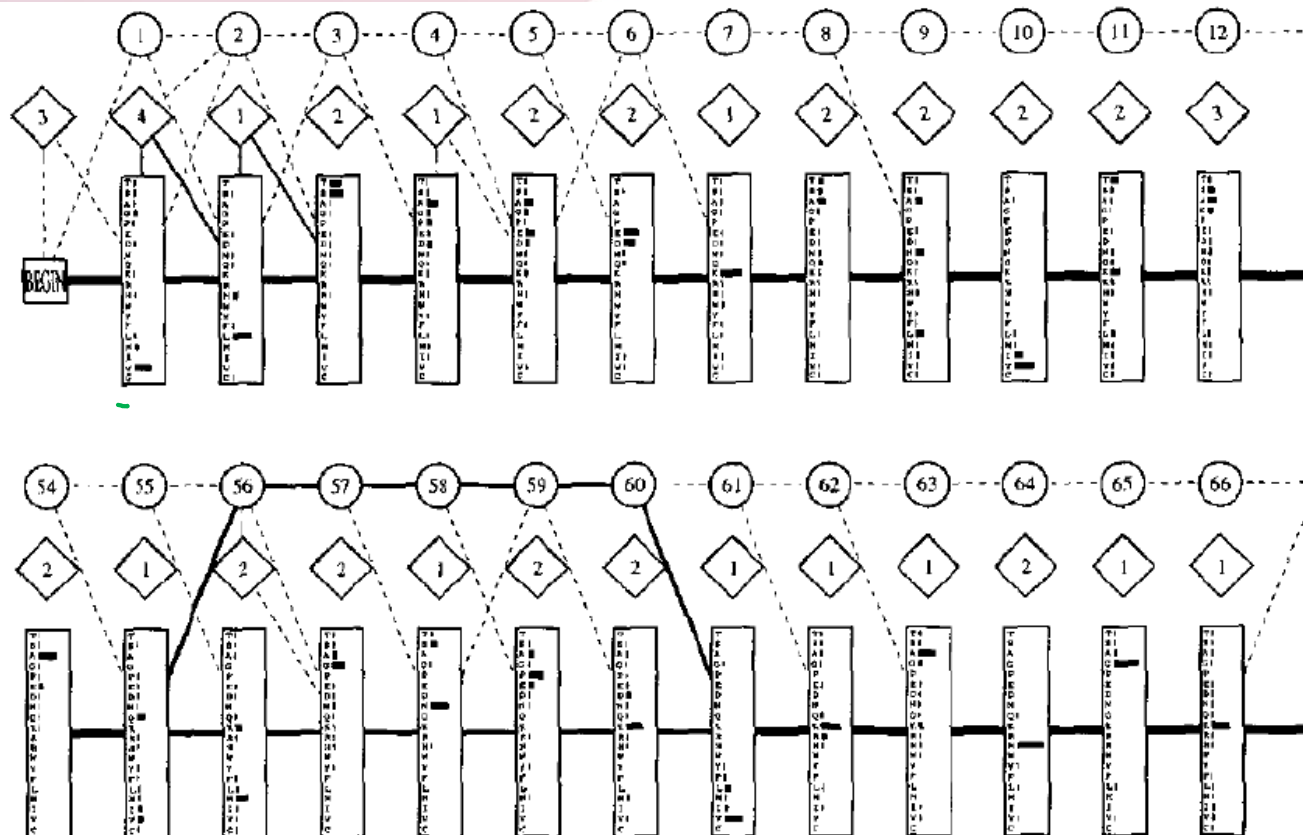
- The HMM model gets trained to derive the optimal parameters.
- Transition probabilities:
 - $\log(a_{MI}) + \log(a_{DI})$ = gap open penalty
 - $\log(a_{II})$ = gap extension penalty
- Emission probabilities:
 - Probability of emitting a symbol a at an insertion state I_j :
$$e_{I_i}(a) = p(a)$$
where $p(a)$ is the frequency of the occurrence of the symbol a in all the sequences.

Estimate Parameters (Learning)



- We want to find a set of parameters to maximize the probability of the observed sequences in the family:
 - maximum likelihood:
$$P(\mathbf{x} \mid \Theta) = P(\mathbf{x}^{(1)} \mid \Theta) \cdot \dots \cdot P(\mathbf{x}^{(m)} \mid \Theta).$$
- Baum-Welch's algorithm (or EM algorithm) (see above slides)

Visualization of Features and Structure in HMM



- Myoglobin protein family. How to interpret it? [Krogh et al. 94]

PFAM – InterPro



- Pfam <http://pfam.xfam.org/> Pfam data and new releases are available through [InterPro https://www.ebi.ac.uk/interpro/](https://www.ebi.ac.uk/interpro/)
- describes ***protein domains***
- Each protein domain family in Pfam has:
 - **Seed alignment**: manually verified multiple alignment of a representative set of sequences.
 - **HMM** built from the seed alignment for further database searches.
 - **Full alignment** generated automatically from the HMM
- The distinction between seed and full alignments facilitates Pfam updates.
 - Seed alignments are stable resources.
 - HMM profiles and full alignments can be updated with newly found amino acid sequences.

PFAM Uses



- Pfam HMMs span entire domains that include both well-conserved motifs and less-conserved regions with insertions and deletions.
- It results in modeling complete domains that facilitates better sequence annotation and leads to a more sensitive detection.

Profile HMM Alignment



- Define $v_j^M(i)$ as the **log-odds** score of the best path for matching $x_1 \cdots x_i$ to profile HMM ending with x_i emitted by the state M_j .
- $v_j^I(i)$ is the log-odds score of the best path ending in x_i being emitted by I_j and
- $v_j^D(i)$ is the log-odds score of the best path ending in state D_j .

Profile HMM Alignment: Dynamic Programming



Global alignment

$$v_j^M(i) = \log\left(\frac{e_{M_j}(x_i)}{p(x_i)}\right) + \max \begin{cases} v_{j-1}^M(i-1) + \log(a_{M_{j-1}, M_j}) \\ v_{j-1}^I(i-1) + \log(a_{I_{j-1}, M_j}) \\ v_{j-1}^D(i-1) + \log(a_{D_{j-1}, M_j}) \end{cases}$$

Compare against
the random model

$$v_j^I(i) = \log\left(\frac{e_{I_j}(x_i)}{p(x_i)}\right) + \max \begin{cases} v_j^M(i-1) + \log(a_{M_j, I_j}) \\ v_j^I(i-1) + \log(a_{I_j, I_j}) \\ v_j^D(i-1) + \log(a_{D_j, I_j}) \end{cases}$$

$$v_j^D(i) = \max \begin{cases} v_{j-1}^M(i-1) + \log(a_{M_{j-1}, D_j}) \\ v_{j-1}^D(i-1) + \log(a_{D_{j-1}, D_j}) \end{cases}$$

For $i = 1, \dots, L, j = 1, \dots, n$

Input seq $x_1 \dots x_L$

Initialization:

$$v_0^M(0) = 0;$$

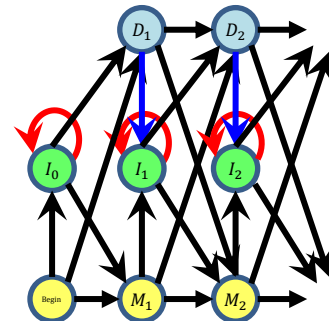
$$v_{j>0}^M(0) = v_0^M(i > 0) = -\infty;$$

$$v_j^I(0) = -\infty$$

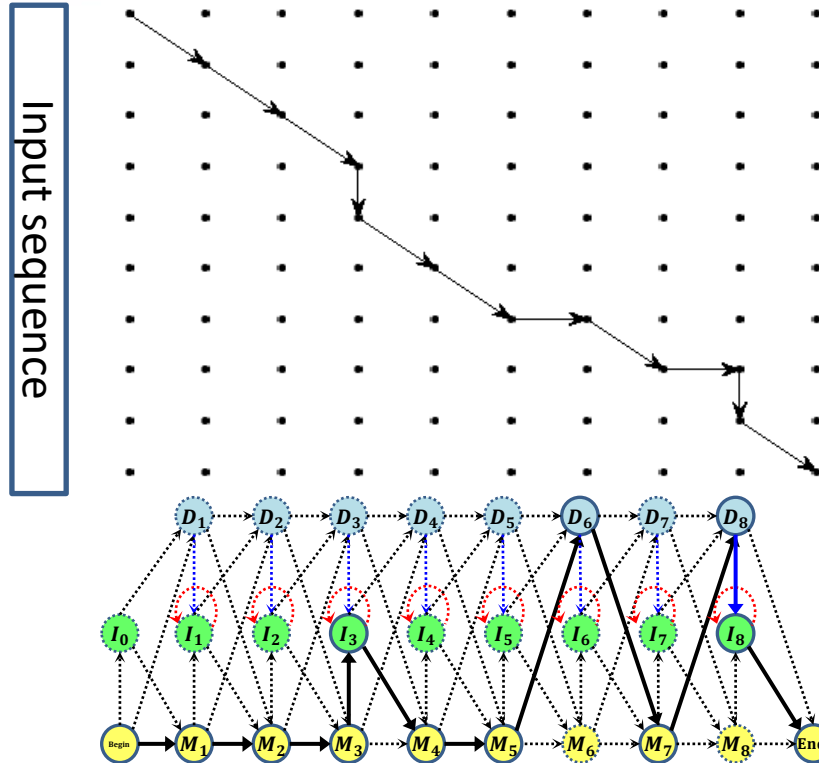
$$v_j^D(0) = -\infty$$

Termination:

$$v = \max[v_L^M(n), v_L^I(n), v_L^D(n)]$$



Paths in Edit Graph and Profile HMM



A path through an edit graph and the corresponding path through a profile HMM

Two Ways to Search



1. Build an HMM for each family in the database. Search a query sequence against the database of HMMs. (Pfam)
2. Build an HMM for a query family, and search HMM against a database of sequences

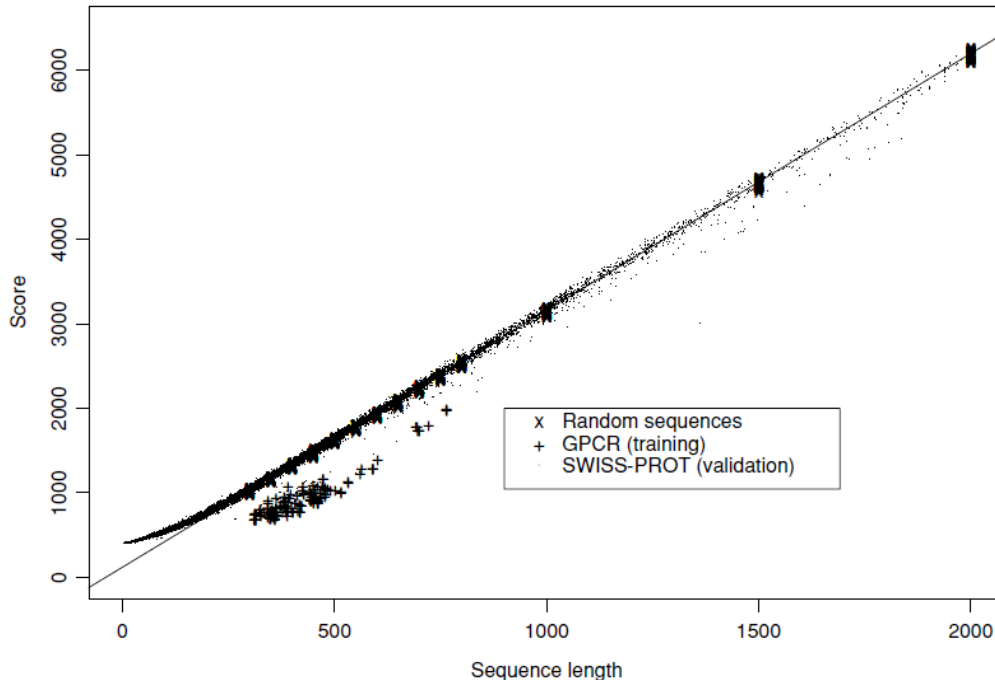
Compute

$P(\text{Sequence} \mid HMM)$



- Forward algorithm to compute $P(x \mid \Theta)$
- We work on: $-\log(P(x \mid \Theta))$: distance from the sequence to the model. (**N**egative **L**og **L**ikelihood score – NLL)
- Unfortunately, $-\log(P(x \mid \Theta))$ is length dependent. So, what can we do?
 - Normalize the Score into Z-score
 - Search the profile against a large database such as Swiss-Prot
 - Plot $-\log(P(x \mid \Theta))$, NLL scores, against sequence length.

Normalize the Score into Z-score

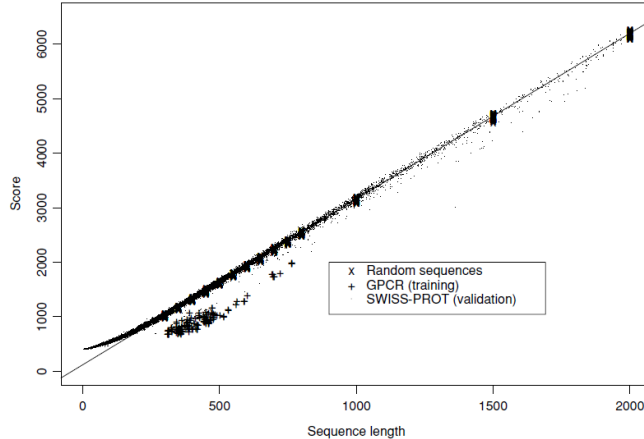


Example: **G-Protein-Coupled Receptors**

- Transmembrane proteins for signaling between environment and a cell

- NLL score is linear with the sequence length.
- NLL scores of the same family is lower than un-related sequences
- We need normalization.

Normalize the Score into Z-score



- Compute Z-score: $\frac{|s-\mu|}{\sigma}$
- $Z > 4$: the sequence is very different from unrelated sequence (for non-database search, a randomization can work)

- NULL model of unrelated sequences:

Length	Mean NLL (μ)	Std (σ)
100	500	5
101	550	6
...		
...		

Application of Profile HMM to Modeling Globin Proteins



- Globins represent a large collection of protein sequences.
- 400 globin sequences were randomly selected from all globins and used to construct a multiple alignment.
- Multiple alignment was used to assign an initial HMM.
- This model then get trained repeatedly with model lengths chosen randomly between 145 to 170, to get an HMM model optimized probabilities.

How Good is the Globin HMM?



- 625 remaining globin sequences in the database were aligned to the constructed HMM resulting in a multiple alignment. This multiple alignment agrees extremely well with the structurally derived alignment.
- 25 044 proteins were randomly chosen from the database and compared against the globin HMM.
- This experiment resulted in an excellent separation between globin and non-globin families.

Outline



1. CG-islands
2. The “Fair Bet Casino”
3. Hidden Markov Model
4. Decoding Algorithm
5. Forward-Backward Algorithm
6. HMM Parameter Estimation
7. Viterbi training
8. Baum-Welch algorithm
9. Applications of HMM in Biology
 - A. Finding genes
 - B. Profile HMM
 - C. **Pairwise Alignment via HMM**

Pairwise Alignment via HMM



Seq 1: A T G R K E
Path : M_1 I_1 I_1 M_2 D_3 M_4 I_4

Seq 2: V C K E R P
Path : M_1 I_1 M_2 M_3 M_4 I_4



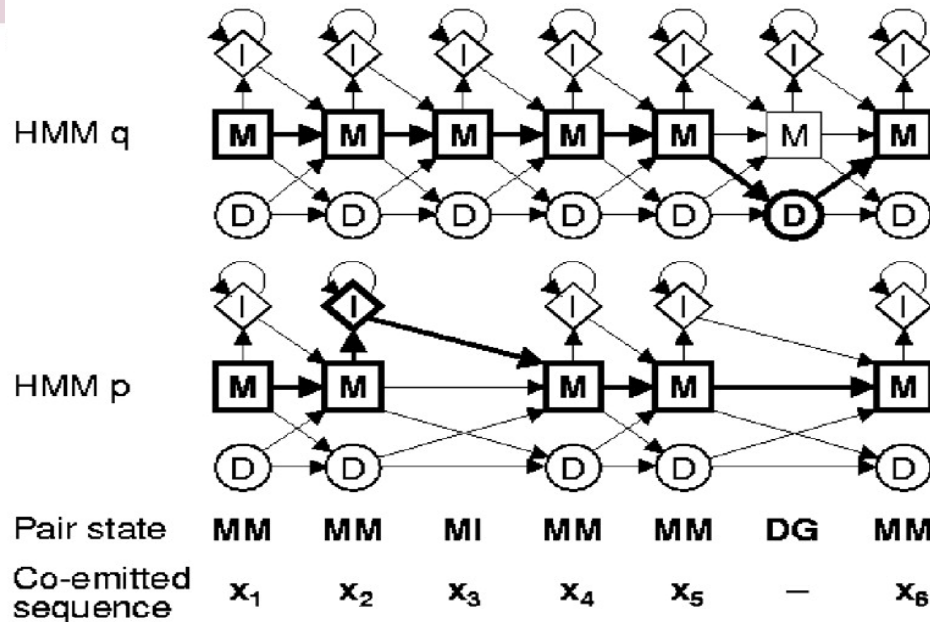
Path: M_1 M_2 M_3 M_4
Seq 1: A T G R - K E
Seq 2: V C - K E R P

HMM for Multiple Sequence Alignment



- Build an HMM for a group of sequences.
- Align each sequence against HMM using Viterbi algorithm to find the most likely path. (dynamic programming).
- Match the main/match states of these paths together.
- Add gaps for delete states.
- For insertion between two positions, use the longest insertion of a sequence as template. Add gaps to other sequence if necessary.

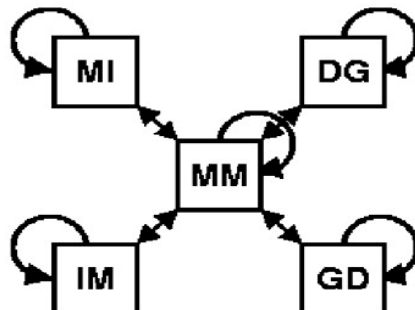
Similarity between HMMs



Generalized sequence
(of states) 1

Generalized sequence
(of states) 2

Allowed
pair state
transitions



Goal: find a sequence of
paired states with
maximum sum of log-
odds scores

COACH Approach



- COACH stands for Comparison Of Alignments by Constructing HMMs
- Given two families of sequences, build a multiple alignment (MSA) for each one of them.
- Build HMM from one MSA
- Align another MSA against the HMM (match each column of amino acids against states in the HMM)
- **How to do Local Alignment:**
 - With respect to sequence: add an insertion state right after the start state and right before the end state.
 - With respect to HMM: start state can jump to any match state and any match state can jump to end state.

HMM Software and Code



- HMMER: <http://hmmer.org> – biosequence analysis using profile hidden Markov models
- The MPI Bioinformatics Toolkit <http://toolkit.tuebingen.mpg.de> many tools **HH**xxxx (based on **HMM-HMM** comparison)
- PRC-HMM – the profile comparer: <http://supfam.mrc-lmb.cam.ac.uk/PRC/>
- COACH: profile-profile alignment of protein families using hidden Markov models : <http://www.drive5.com/lobster/>
- HMMCOMP – HMM-HMM comparison: <http://users-cs.au.dk/cstorm/hmmcomp/>
- MUSCLE – multiple alignment software: <http://www.drive5.com/muscle/>